CS-214

Day: Date: / /

$a\{\}$

a

+ is mandatory
* is optional
↳ start with len = 0 upto N

$a^* b^*$ → if you choose b first, you can't choose a next ??

$L = \{x^{odd}\}$

$= \{x \quad xxx \quad xxxxx \ldots\}$

$x(xx)^*$

$(a+b)^*$

Λ, choose 'a OR b, ', ', ', ', ', ', ...
len 0 len 1 len 2 len 3 len N

Example,

$a(a+b)^*$

aa^*b^* $a(ab)^*$

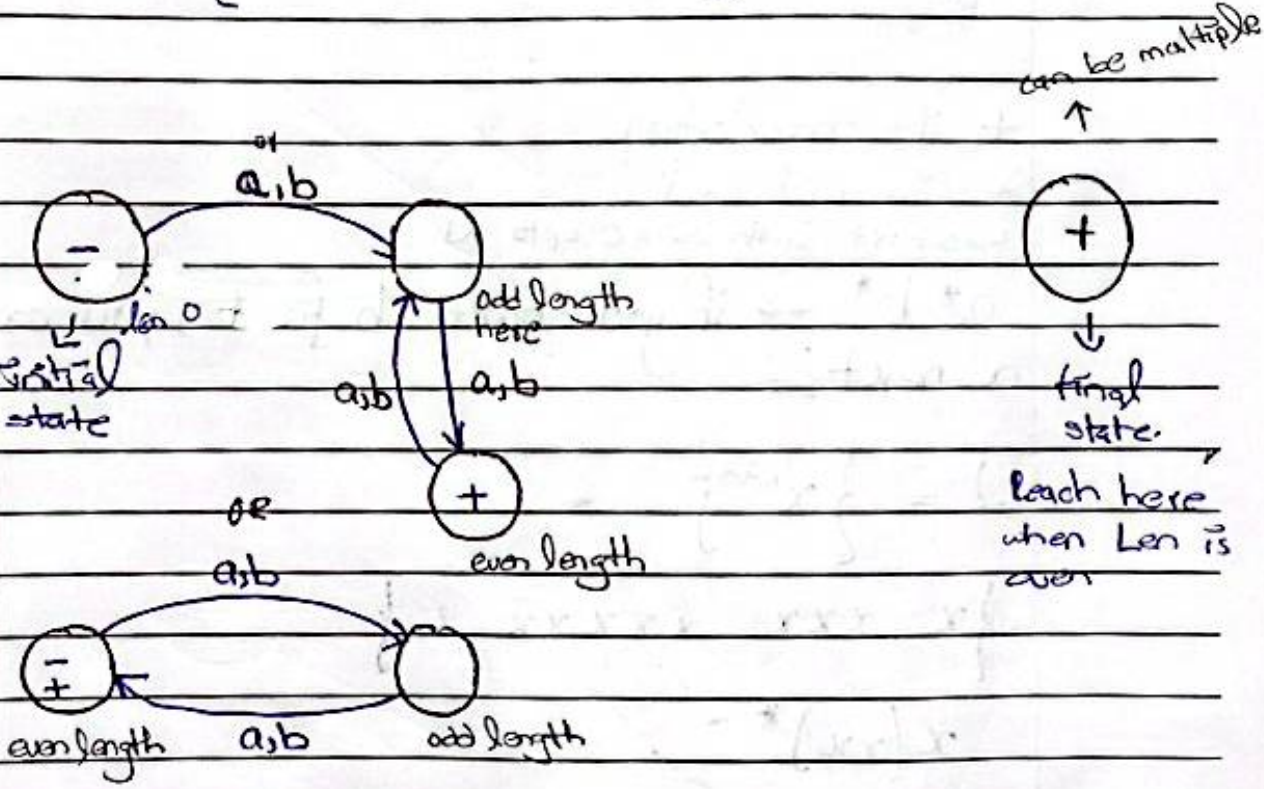
limited to a must come before b

GALAXAY

Day: Date: / /

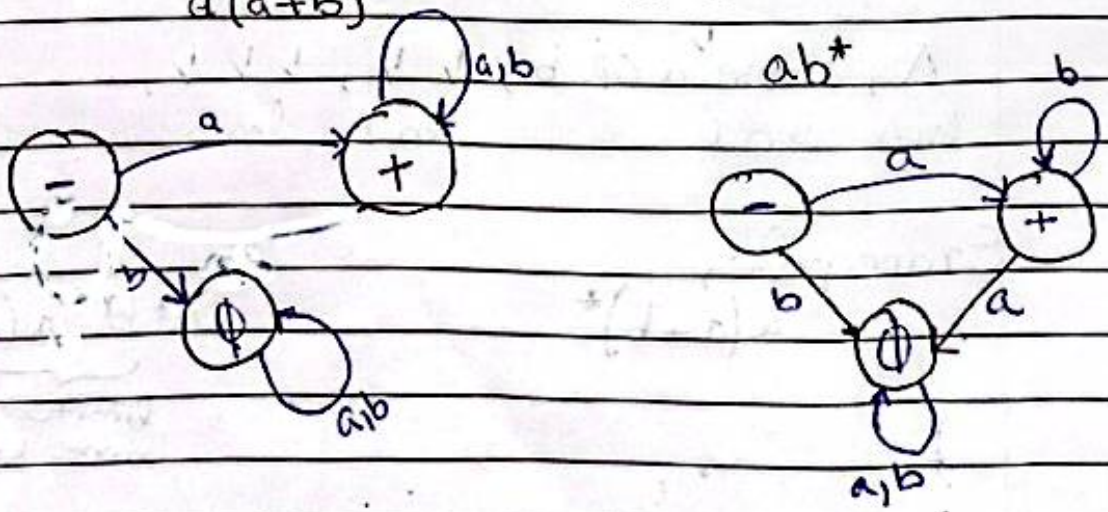
(DFA) Finite Automata → Finite states & executes according to the input

$L = \{$ All words of even length $\}$



∴ All the words in the language are represented

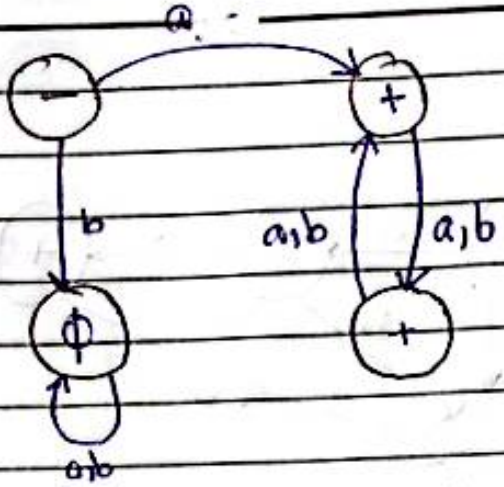
$a(a+b)^*$



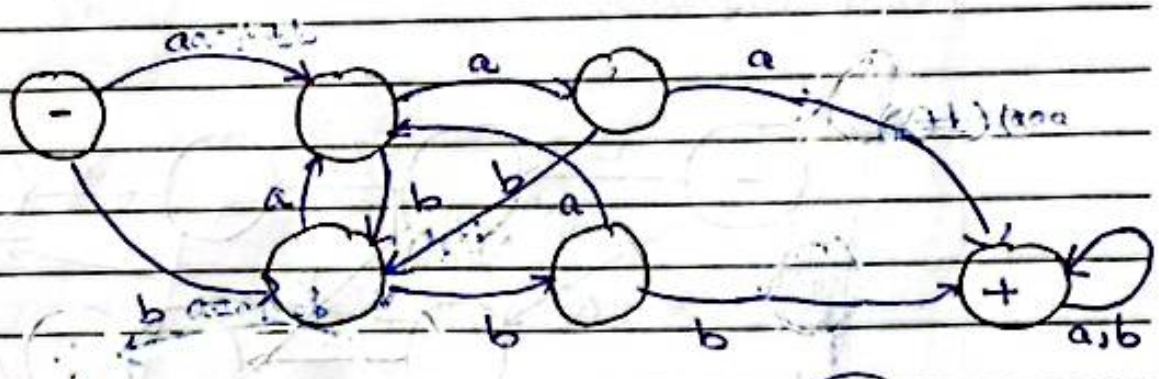
ab^*

GALAXAY

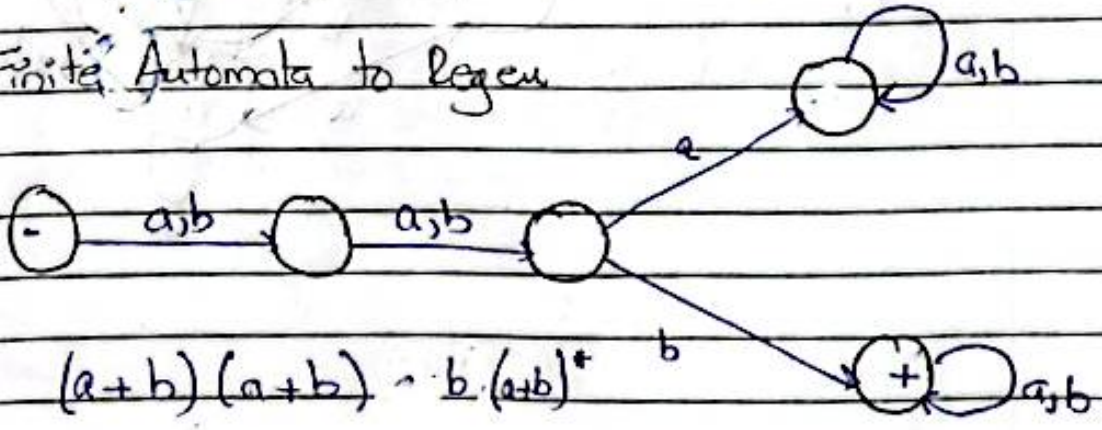
Day: Date: / /



Build DFA that accepts all words containing a triple letter either aaa or bbb.



Finite Automata to Regex

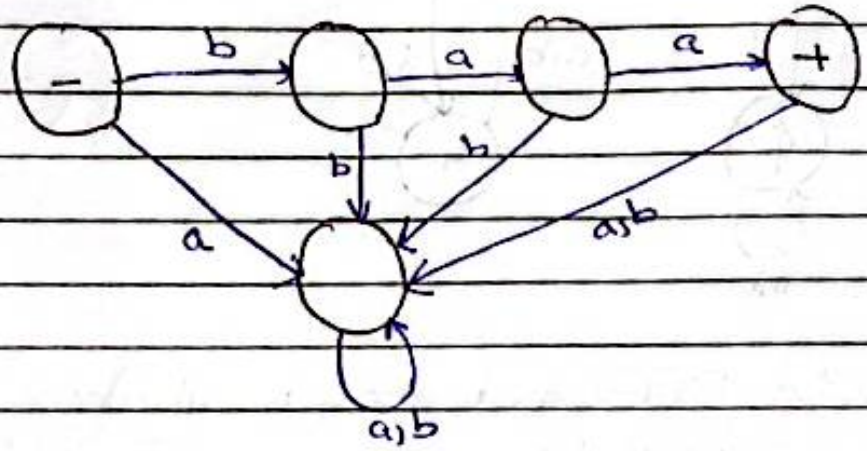


$(a+b)(a+b)\cdot b(a+b)^*$

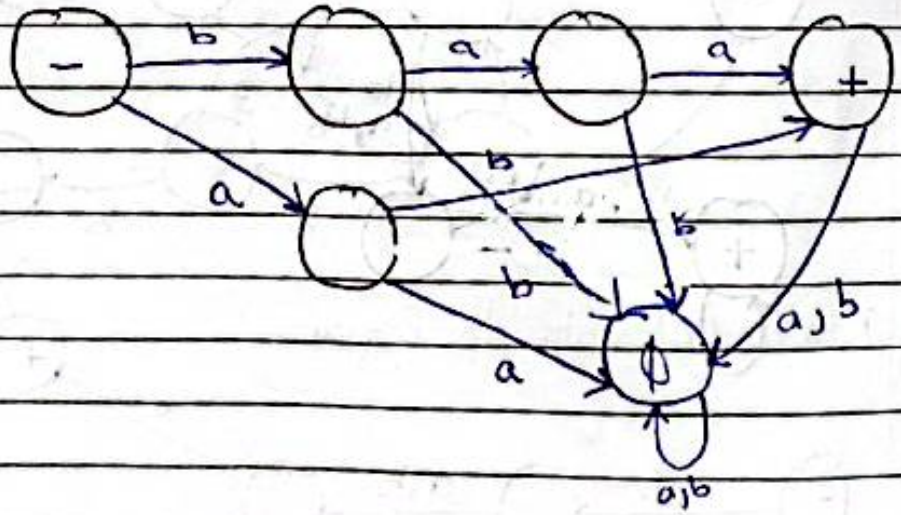
third letter is 'b'.

Day: Date: / /

Accept baa

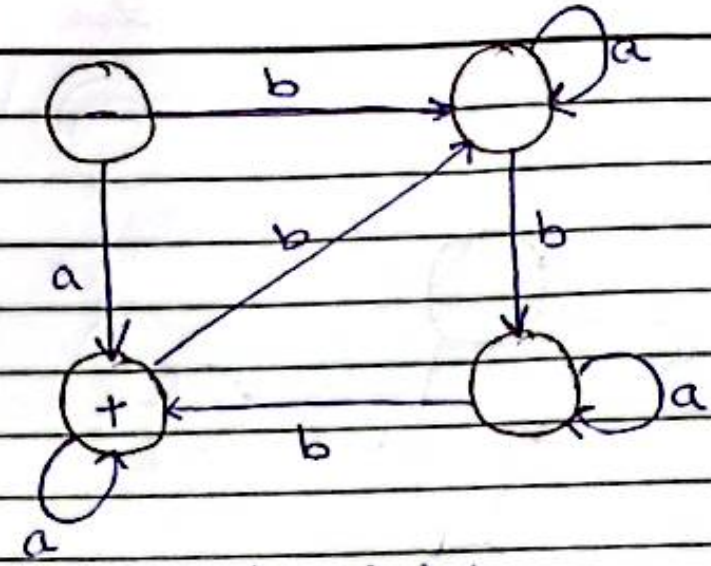


Now, ab also



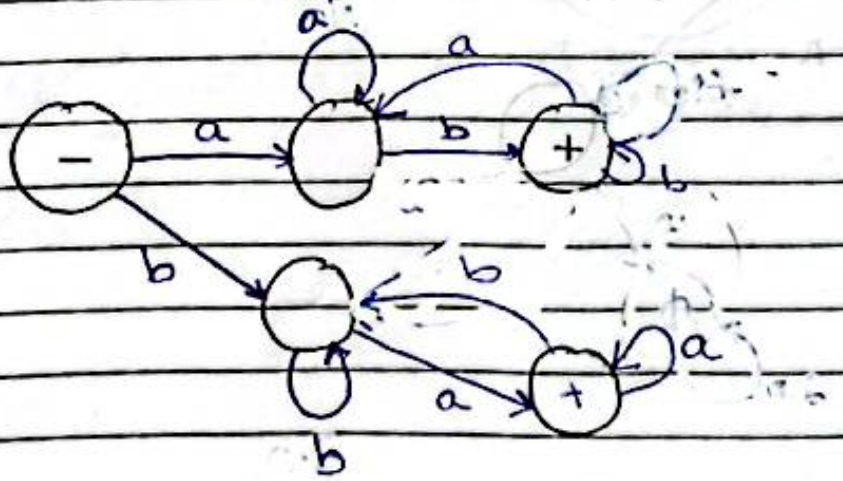
GALAXAY

Day: Date: / /



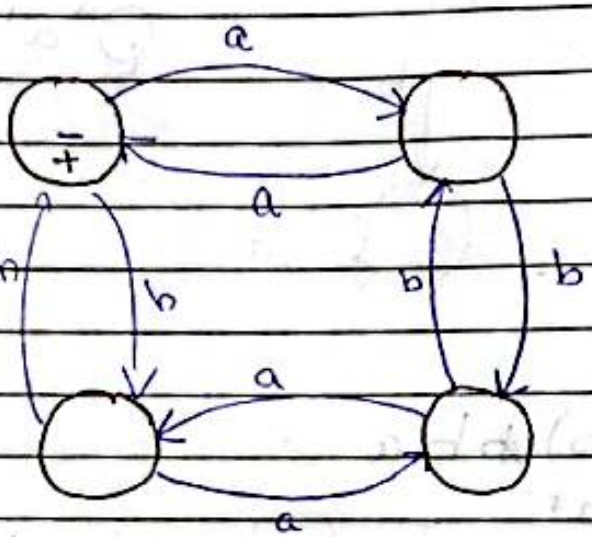
$$\left[ba^*ba^*ba^* + aa^*\right]^+$$

Start & end must be different

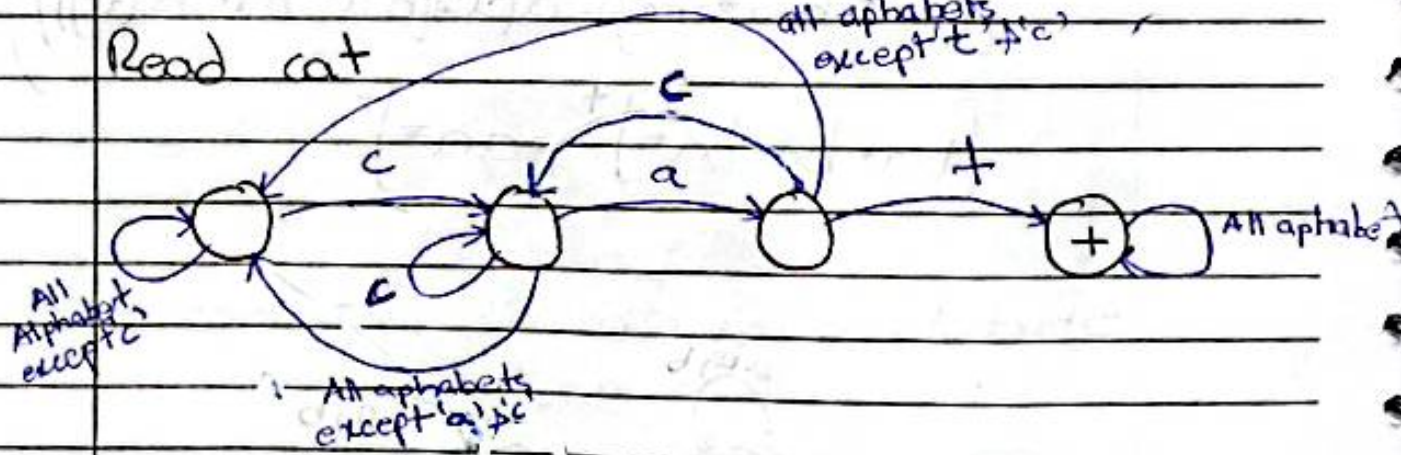


Day: Date: / /

Even Even

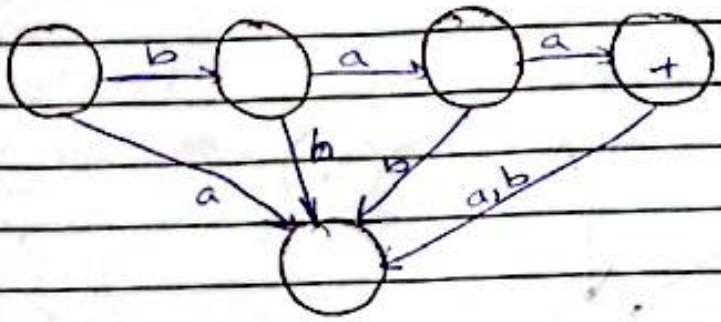


Read cat

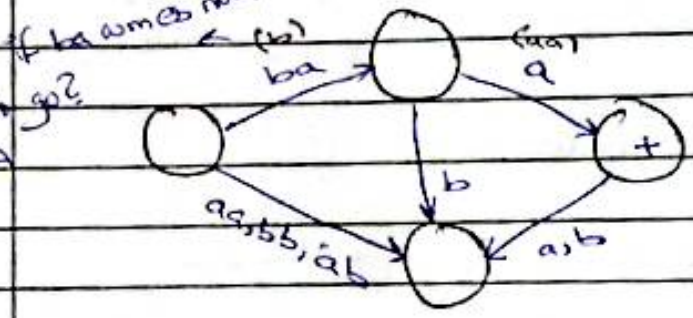


GALAXAY

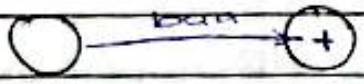
Day: Date: / /



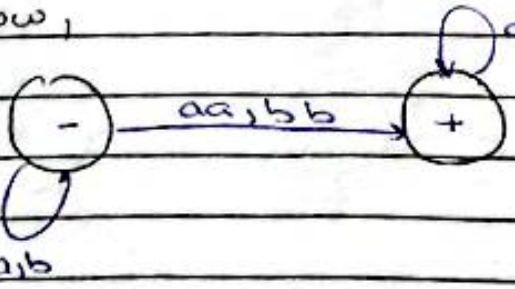
→ Every transition exists
→ Every transition path is unique



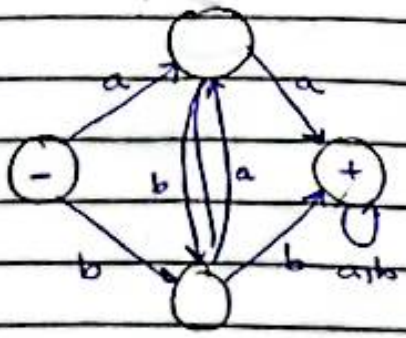
↳ We have to change our definition of rejection since every transition can't exist?



Now,

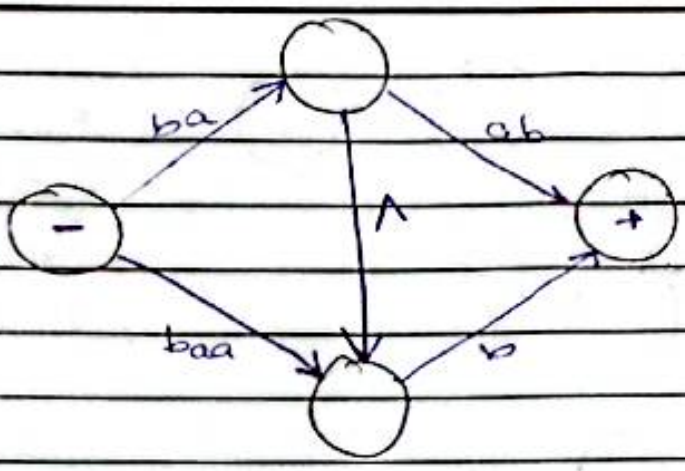


→ aa → accepted
a a → Rejected



Rejection criteria:
(1) Input does not end in final state
(2) Machines crashes

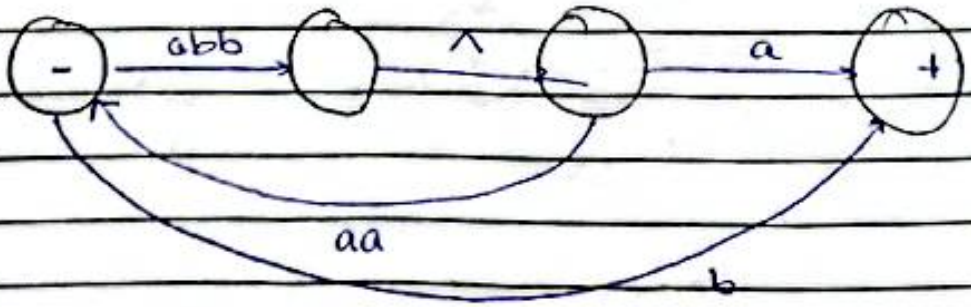
Day: Date: / /



baab
if there is one path for above that leads to the final state the word is accepted.

Now,
bab is also accepted.

Finite states
Multiple −, +
Σ.
Finite set of transitions (Any number of letters including Λ).



GALAXAY

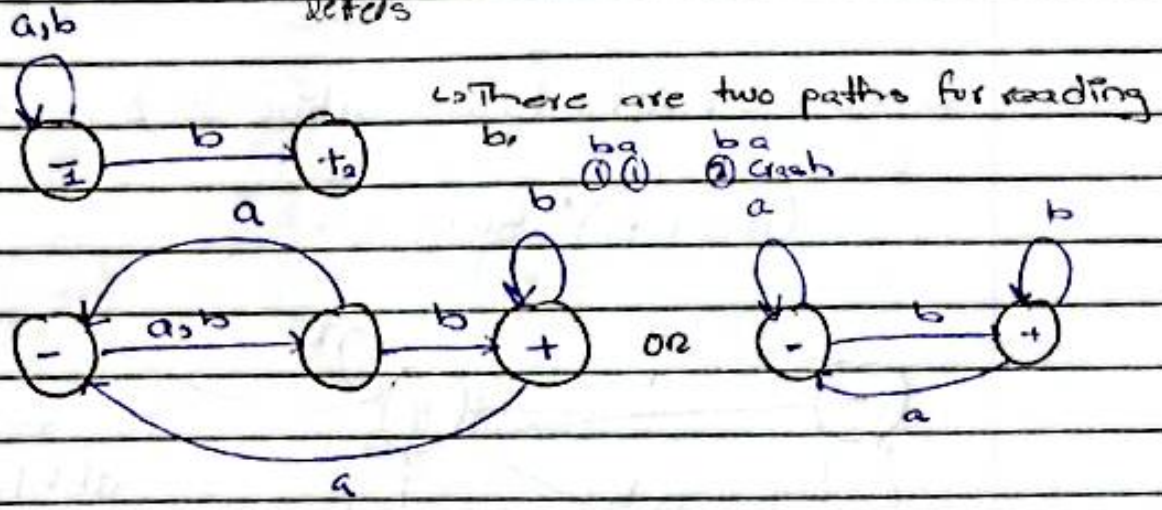
Day: Date: / /

Rule:

~~Read only one letter~~
~~Can't crash~~
~~(No transitions)~~

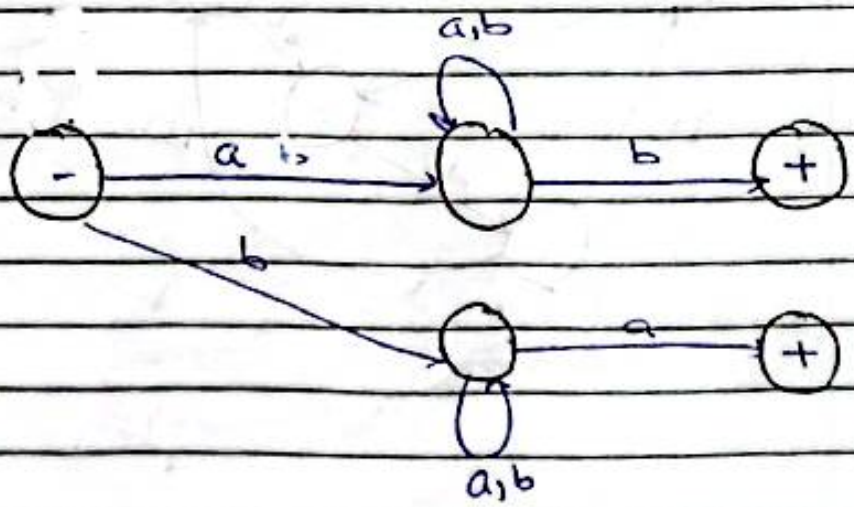
↳ Each FA is a transition graphs
But All TGs are NOT satisfying definition of FA.

(^ transitions)
Can crash
Read one or more letters

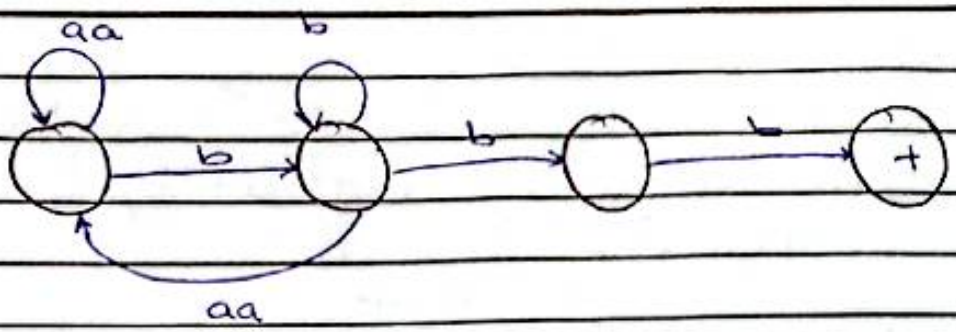


↳ There are two paths for reading ba

All words starting & ending with different letters.

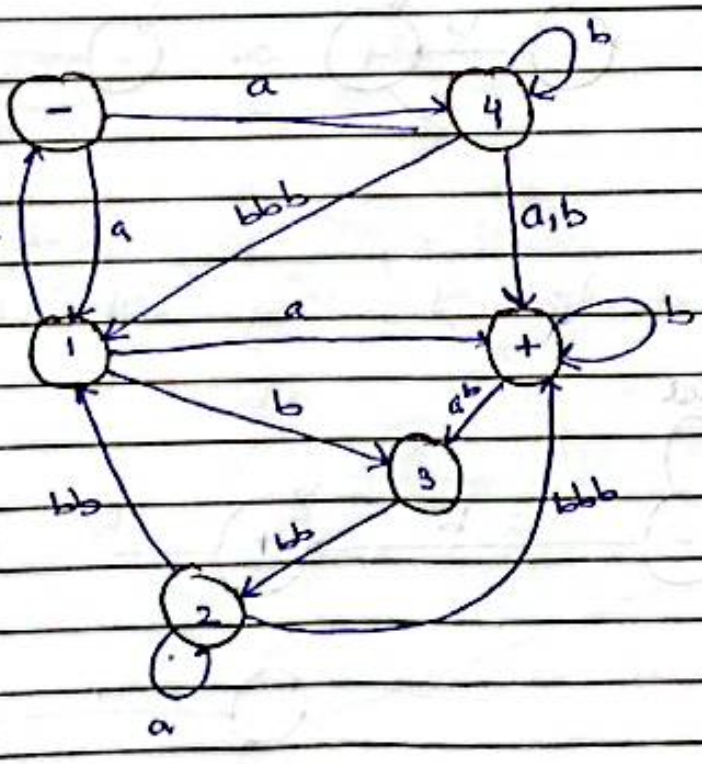


Day: Date: / /



Language : Atleast 3 consecutive b & even no. of as.

$(aa+b)^* bbb$



is acceptable by:

abbbabbbabba

YES

44++3221+

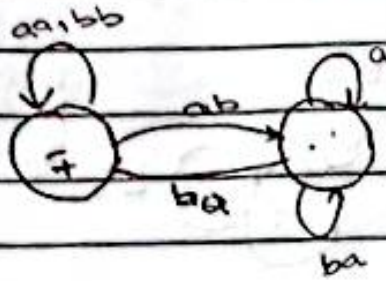
-4+++3221+

-444+3221+

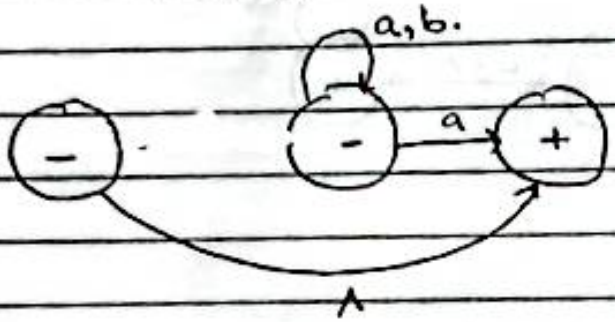
GALAXAY

Day: Date: / /

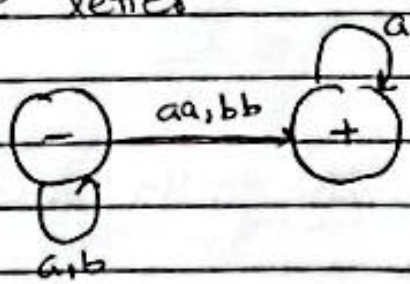
Even - Even language



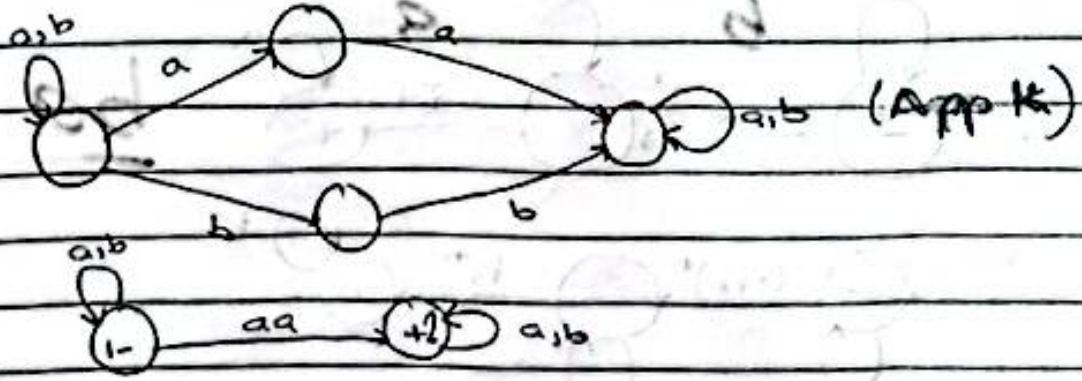
$\Lambda + (a+b)^* a$



Double letter



(Mera)

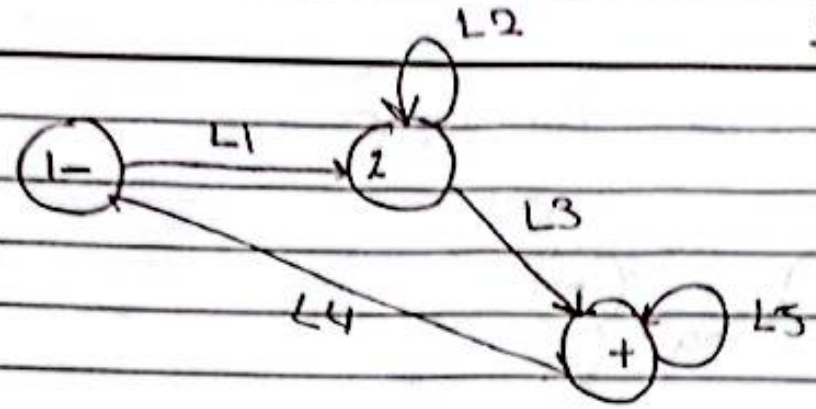


(App k)

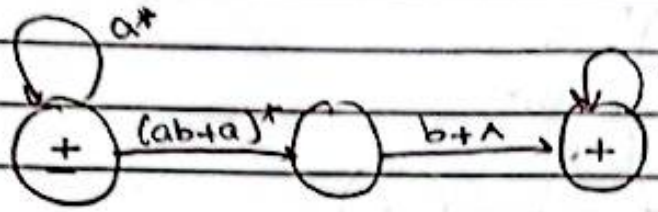
GALAXAY

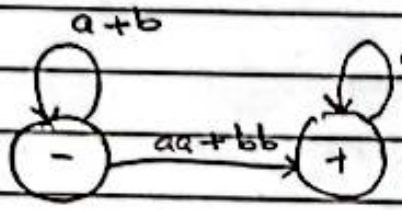
Generalized TG - (GTG)

Day: Date: / /

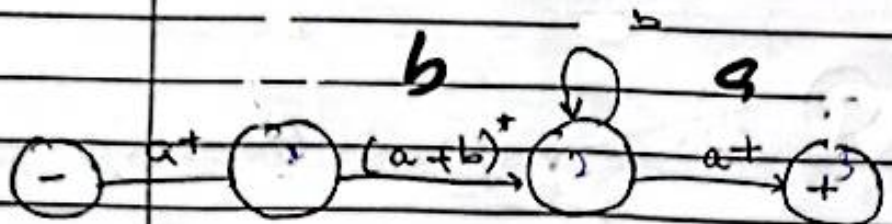


This includes Regex in transition instead of a/ multiple letters.

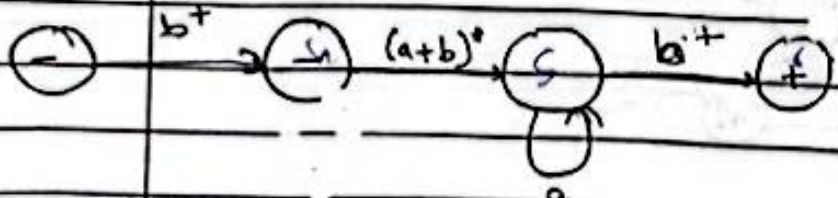




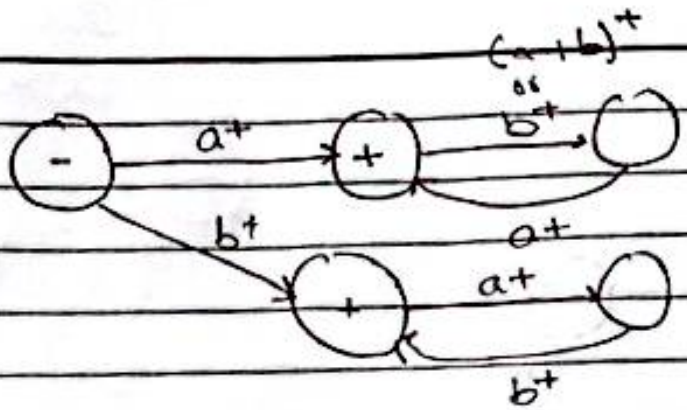
L = {Beginning & ending with same letter}



ba



Day: Date: / /

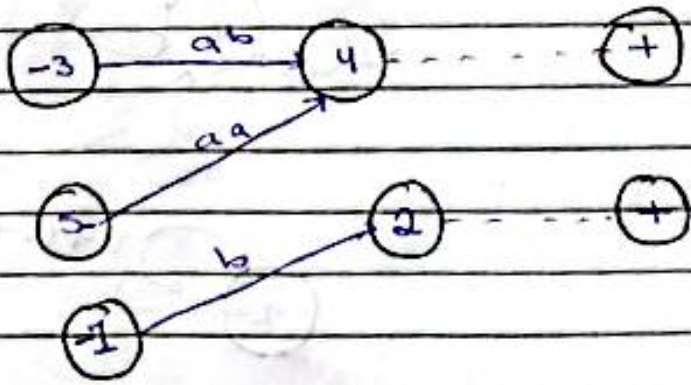


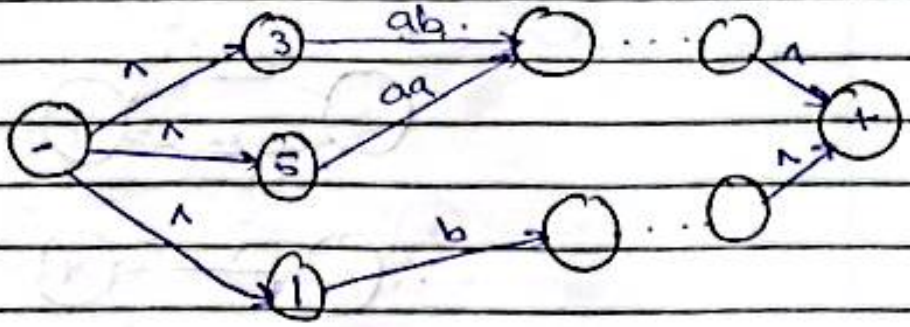
Kleene's theorem

↳ FA → TG → Regex

① Simplification

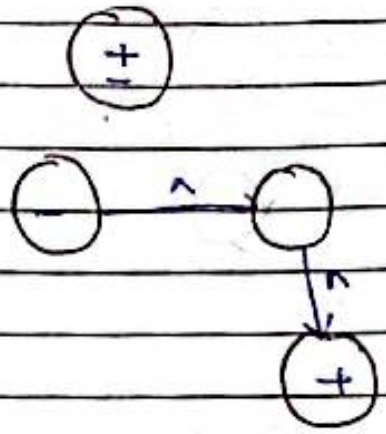
↳ Must have one initial and final state





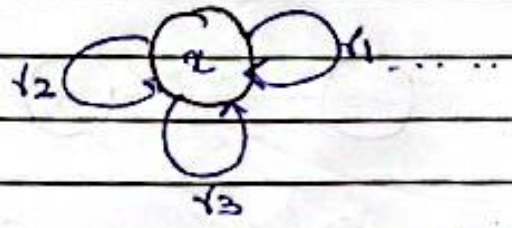
GALAXAY

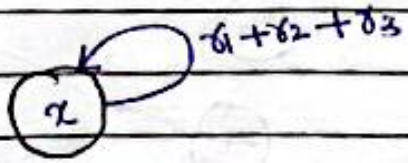
Day: Date: / /



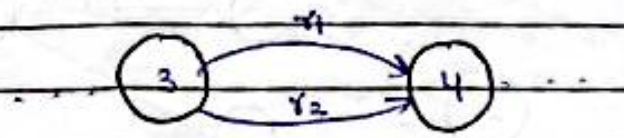
(2) Build R.E

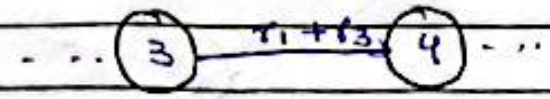
↳ Dealing with self loops





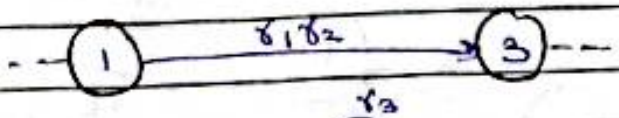
↳

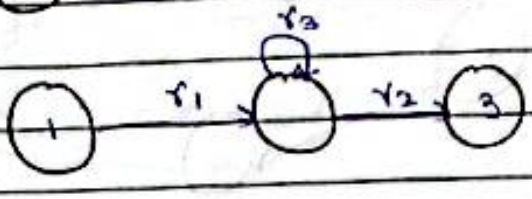


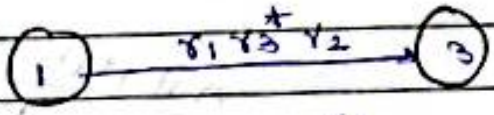


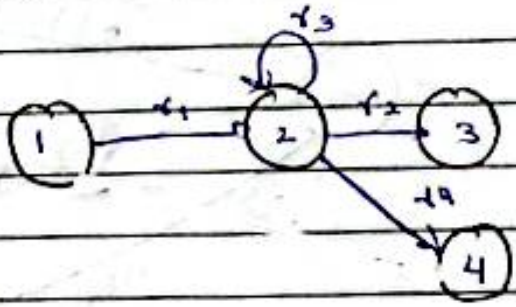
Day: Date: / /

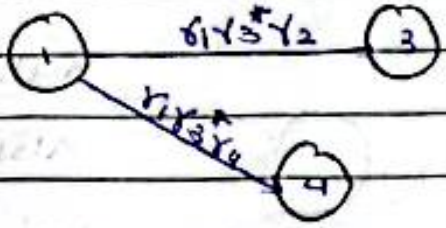
(3) Bypass state



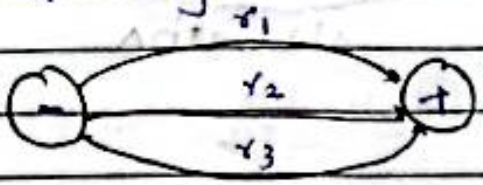


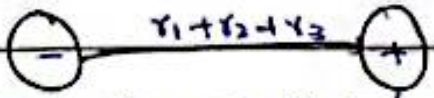






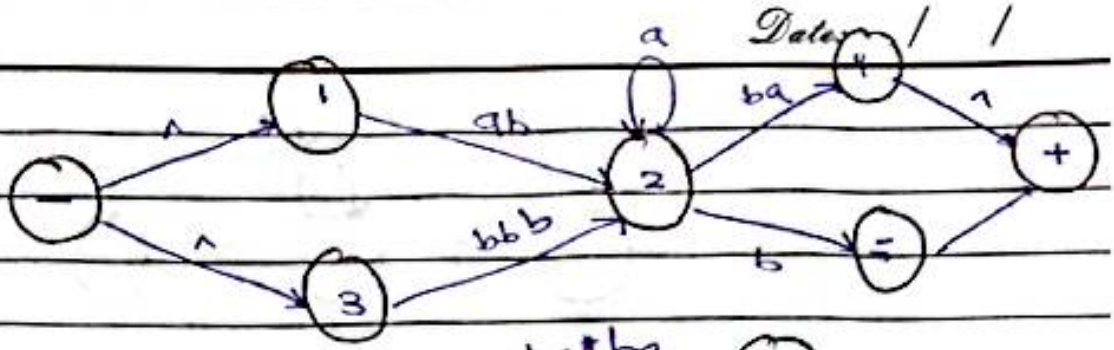
Do this repeatedly

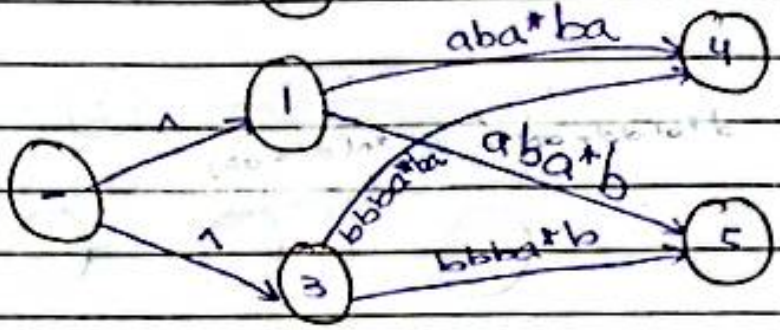


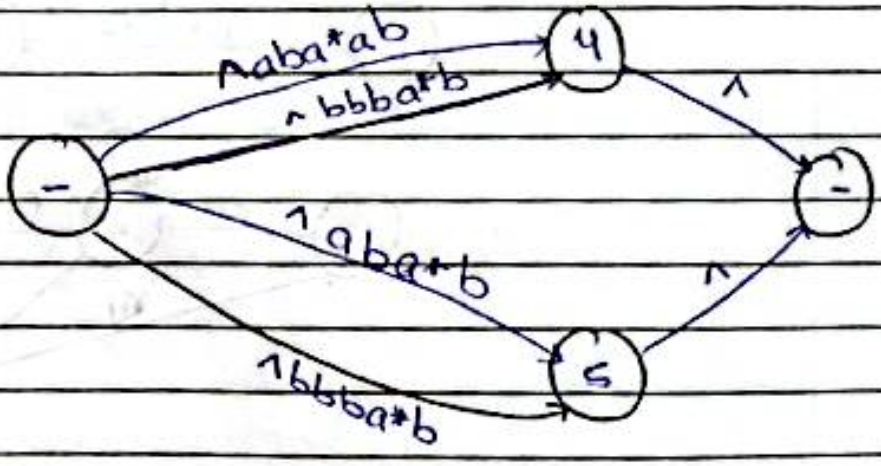


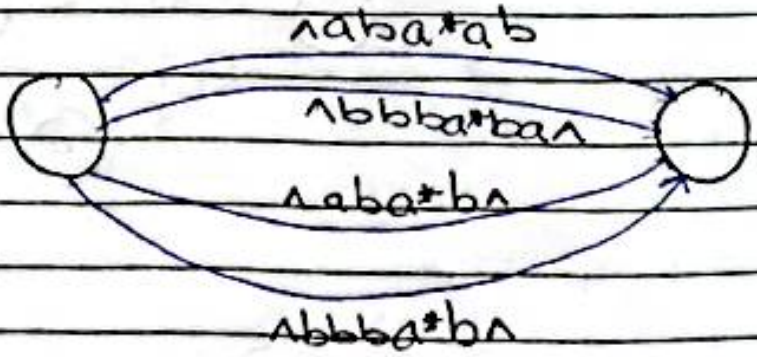
when reach only intial & final state then its Regex

Day: Date: / /



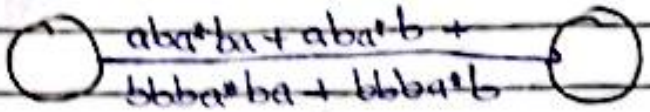




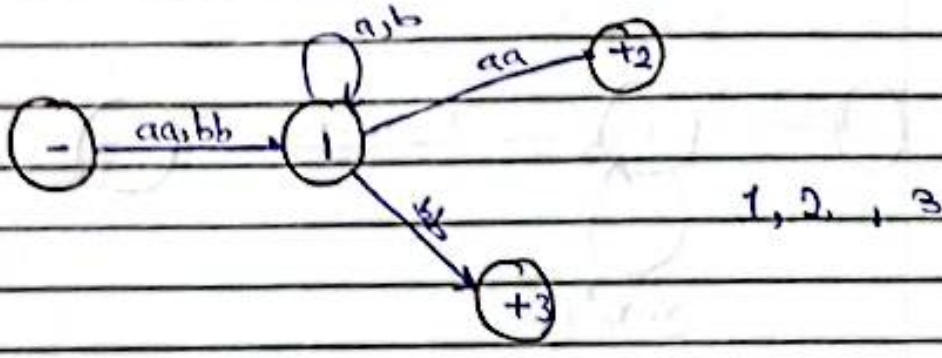


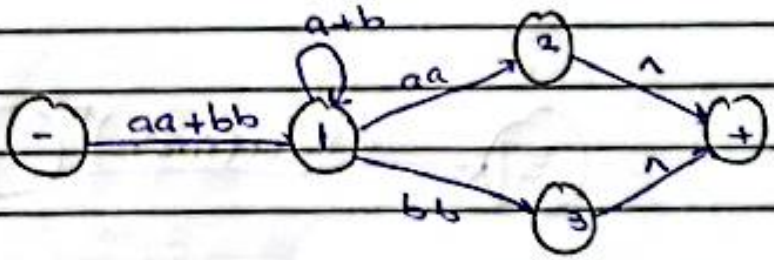
GALAXAY

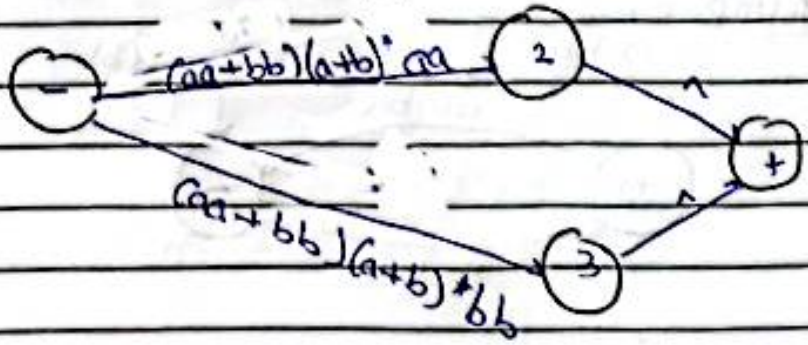
Day: Date: / /

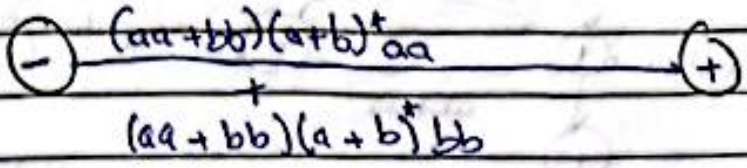


Example,





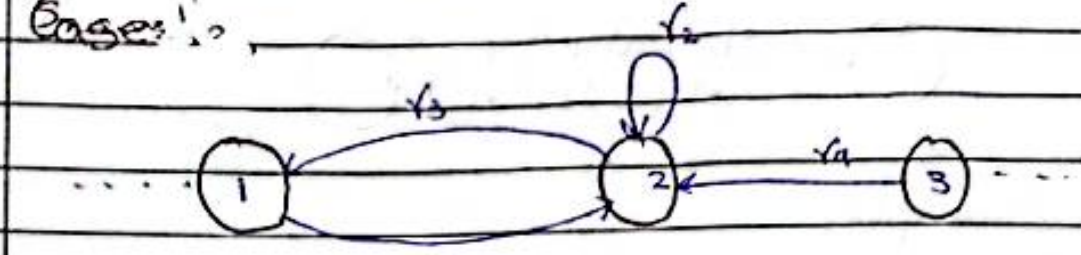


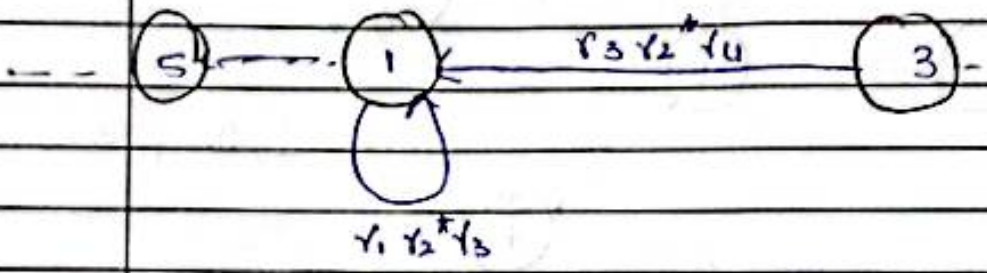


GALAXAY

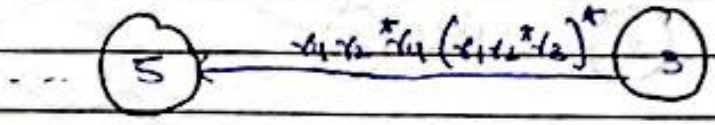
Day: Date: / /

Cases:-

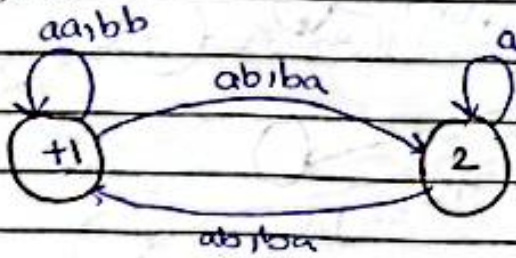




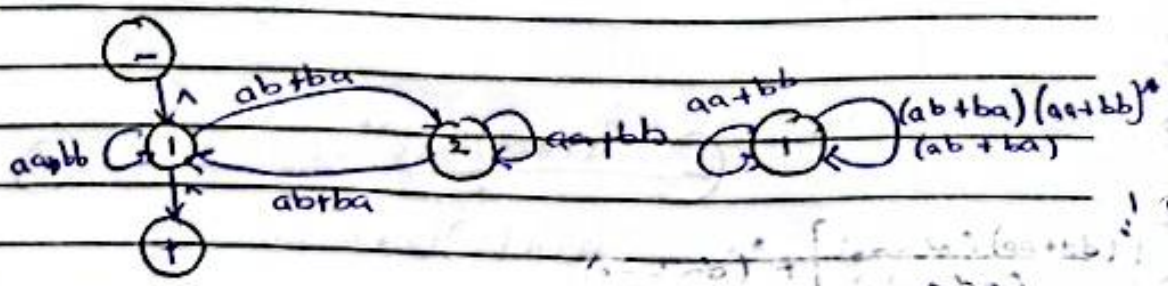
for

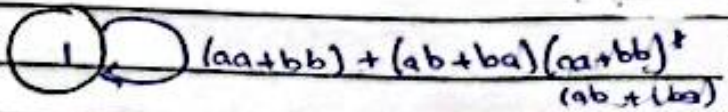


Example:



2, 1



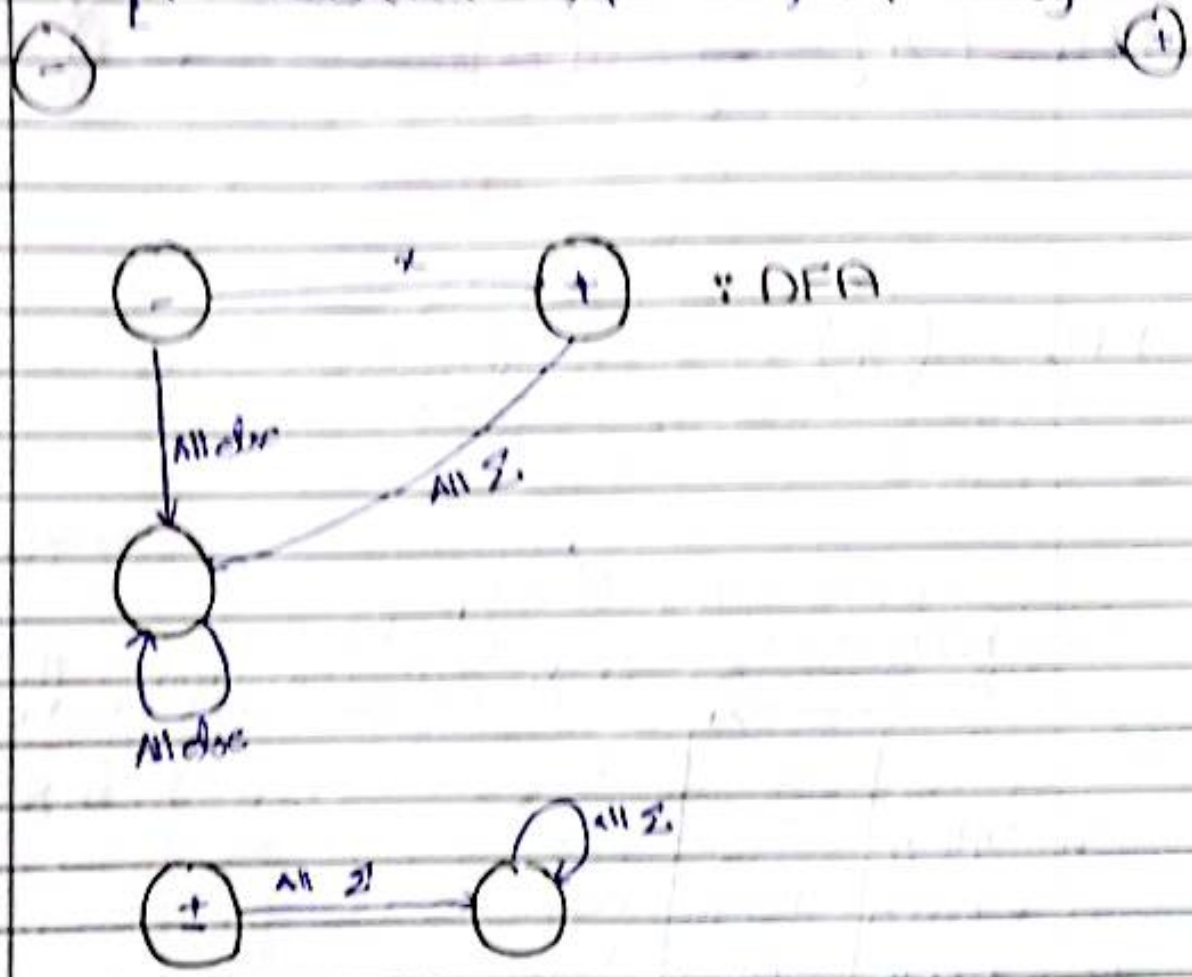


GALAXAY

Page

Date / /

$[(a+b+ba)(aa+bb)^*(ab+ba)+(aa+bb)]^*$



$FA_1 \longrightarrow r_1$

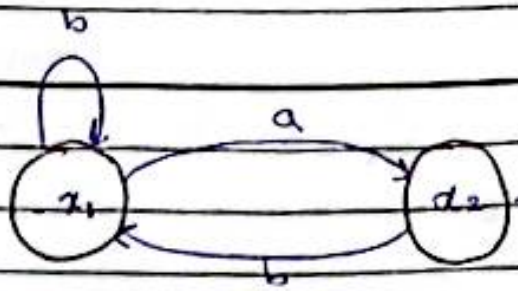
$FA_2 \longrightarrow r_2$

$FA_3 = r_1 + r_2$

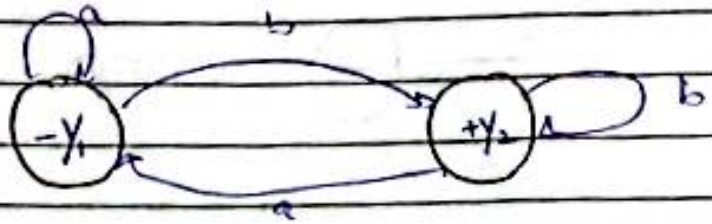
GALAXAY

Day: Date: / /

FA1



FA2



$-Z_1 = -x_1$ or $-y_1$

From Z_1 read a, Z_2
$a \rightarrow x_2$ or y_1 transition from both FA for 'a' from initial state

	a	b	$+Z_5 = x_3$ or y_2	$+Z_4$	$+Z_5$
$-Z_1 = -x_1$ or $-y_1$	Z_2	$+Z_3$			
$Z_2 = x_2$ or y_1	$+Z_4$	$+Z_3$			
$+Z_3 = x_1$ or y_2	Z_2	$+Z_3$			
$+Z_4 = x_3$ or y_1	$+Z_4$	$+Z_5$			

From Z_2 read b:

$b \rightarrow x_1$ or $y_2 = +Z_3$

We read b from FA2 we will reach +. So, Z_3 will also be +.

From Z_3 read a:

$a \rightarrow x_3$ or y_1

$b \rightarrow x_1$ or y_2

From Z_3:

$a \rightarrow x_2$ or y_1

$b \rightarrow x_1$ or y_2

From Z_4:

$a \rightarrow Z_3$ or Y_1

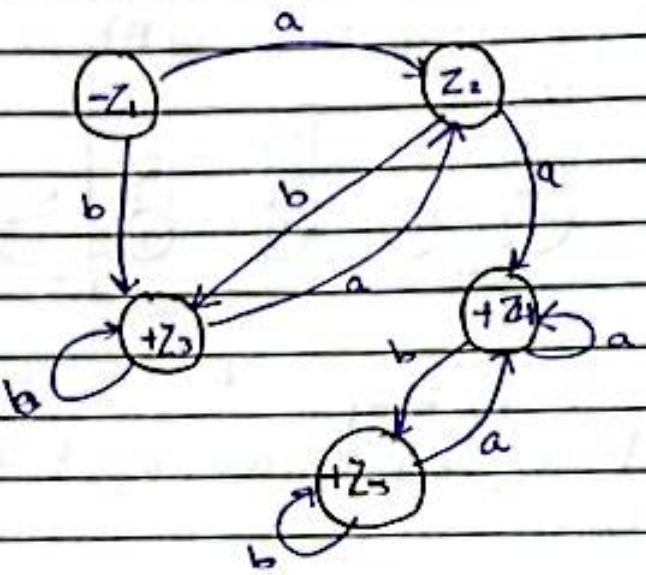
$b \rightarrow x_3$ or y_2

GALAXAY

Day: Date: / /

Now,

NFA can read only one letter.
TGs can read more than one letter.

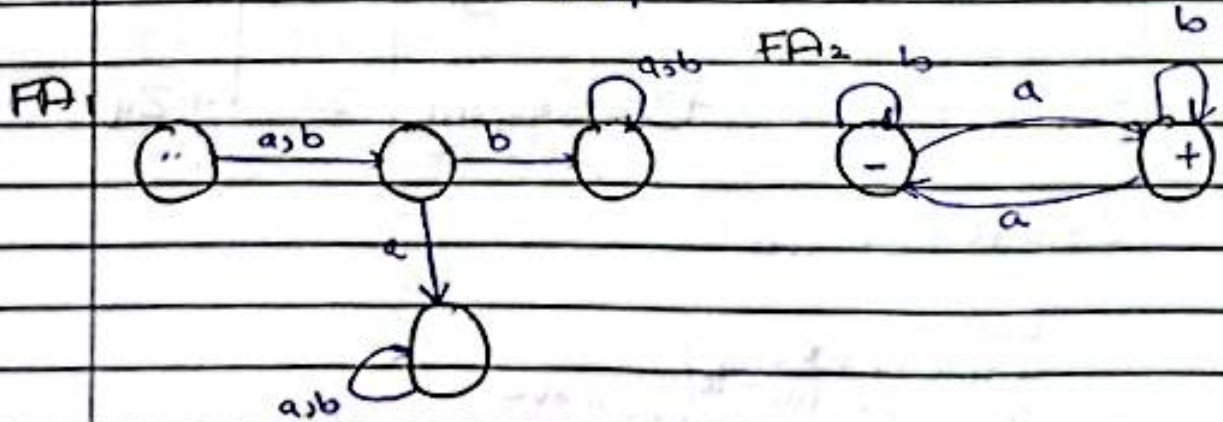


Rule 3

$FA_1 \longrightarrow r_1 = a$ $FA_2 \longrightarrow r_2 = b$

$FA_3 \longrightarrow r_2 r_2$

$r_2 r_1$



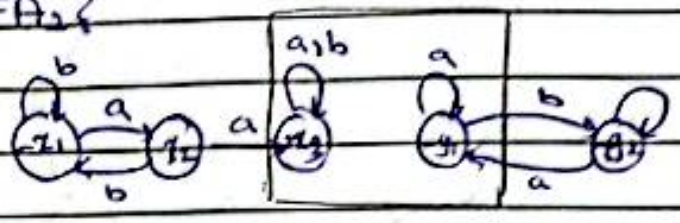
$FA_3 = FA_1 FA_2$

GALAXAY

Day: Date: / /

abab bab
L1 L2

How to know
Who controls/when to stay at + of FA_1 & when to move forward to FA_2?



abbbab
FA_1 FA_2

in FA_1

How many iterations would leave input that can be read in FA_2.

Naw, algorithm:

$FA_3 = FA_1 FA_2$
↙ initial from here

	a	b
$Z_1 = -x_1$	Z_2	Z_1
$Z_2 = x_2$	Z_3	Z_1
$Z_3 = x_3$ or y_1	Z_3	$+Z_4$
$-Z_1 = -x_1$	Z_3	$+Z_4$
$+Z_4 = x_3$ or y_1 or y_2		

$Z_3 = +x_3$ or y_1
↙ read a
reach $+x_3$
So now we have to/can transition on $+x_3$ or move to y_1 so when writing we write initial of FA_2 as well

At Z_3:
on reading a $= x_3$ or y_1 or $y_1 \Rightarrow x_3$ or y_1

At Z_3:
on reading b:
$+Z_4 = x_3$ or y_1 or $+y_1$

GALAXAY

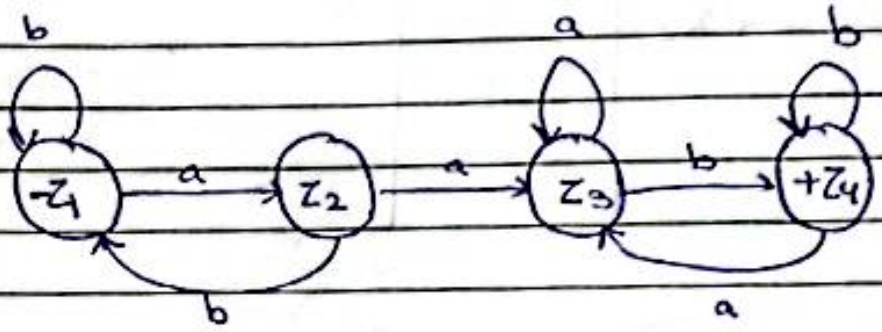
Day: At Z_4: Date: / /

At reading a :

$x_3 \text{ or } y_1 \text{ or } y_1 \text{ or } y_1 = x_3 \text{ or } y_1$

At reading b:

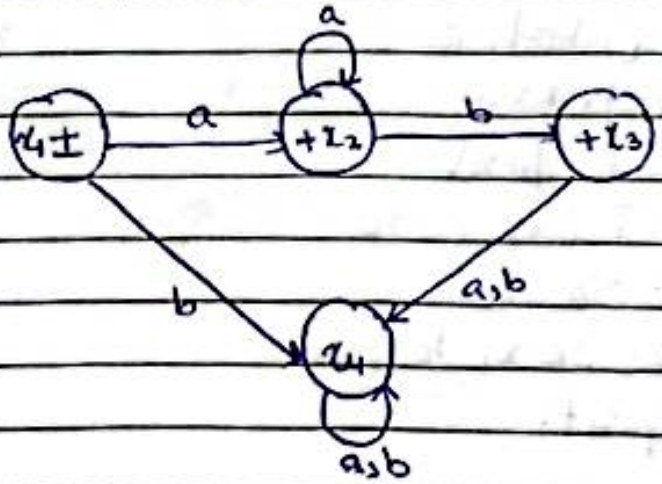
$x_3 \text{ or } y_1 \text{ or } y_2 = +Z_4$



FA_1 accepts $\rightarrow \gamma_1$

FA_2 accepts $\rightarrow \gamma_1^*$

$\gamma = a^* + aa^*b$



GALAXAY

Day: Date: / /

$r^* = (a^* + aa^*b)^*$

Now,

	x_1	a	b
$\pm Z_1 = x_1 \text{ OR } x_1$		Z_2	Z_3
$+Z_2 = x_2 \text{ OR } x_1$		Z_4	
$Z_3 = x_4$			
$Z_4 = +x_2$			

$(a^* + aa^*b)^*$

a

~~we are here after reaching *. Now we~~ can or cannot repeat so we add or like Z_2 state above.

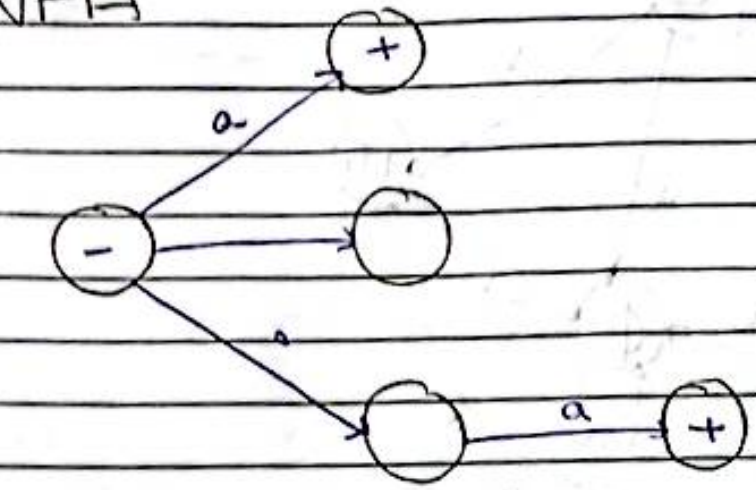
chosen on first iteration on reading a on x_1 to reach x_2 (which is final state). Now since this is final state now I can redo use of *. So we add OR x_2 OR x_1 to show we can repeat.

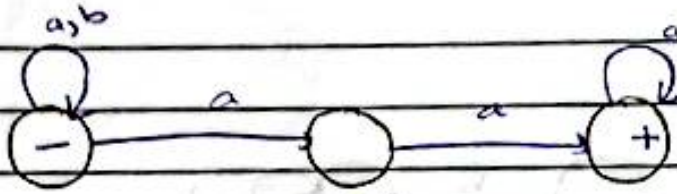
x_2

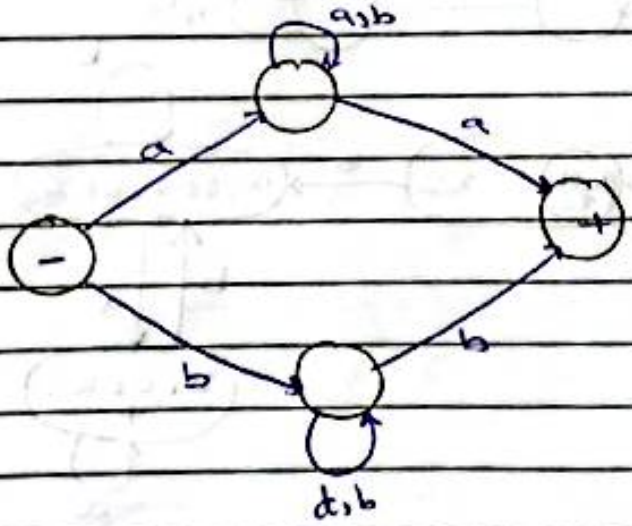
GALAXAY

Day: Date: / /

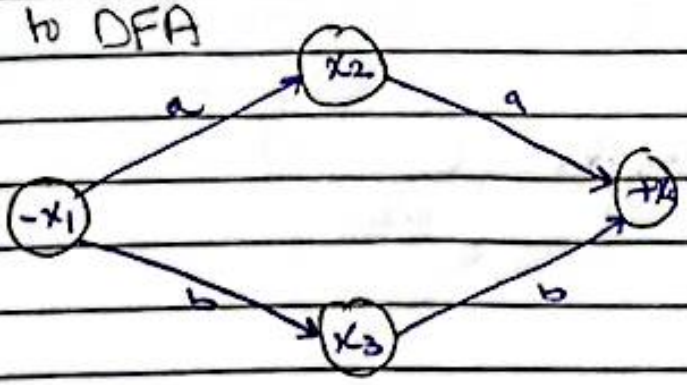
NFA





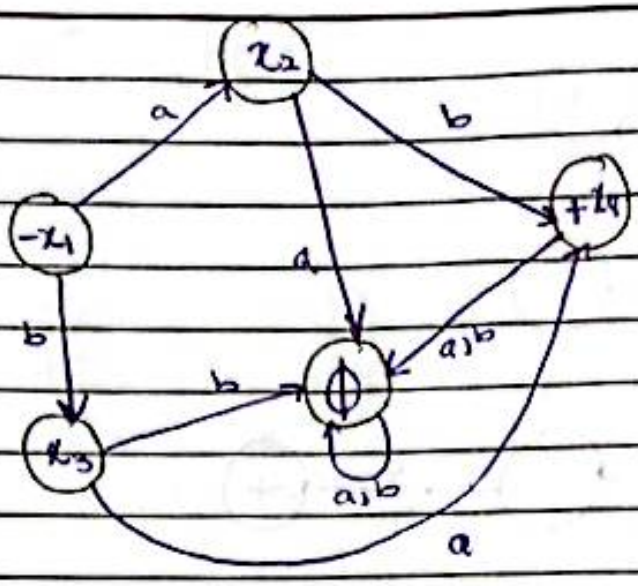


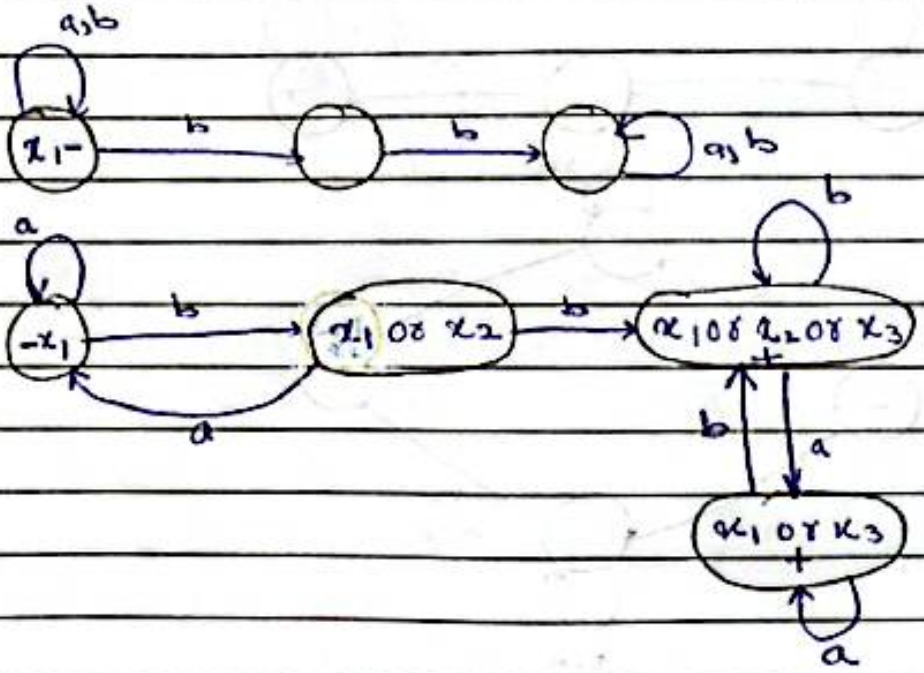
NFA to DFA



GALAXAY

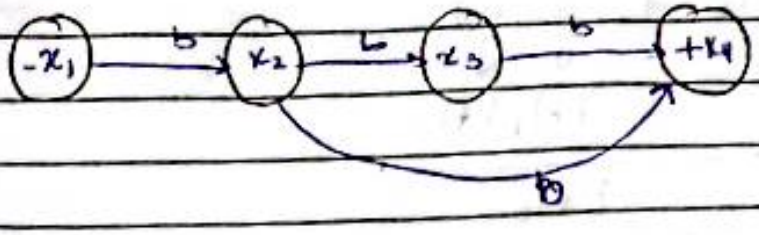
Day: Date: / /

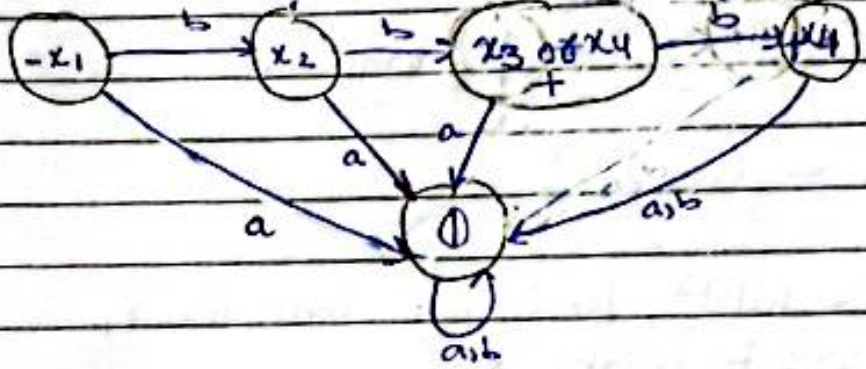




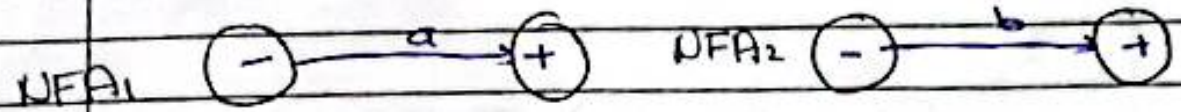
GALAXAY

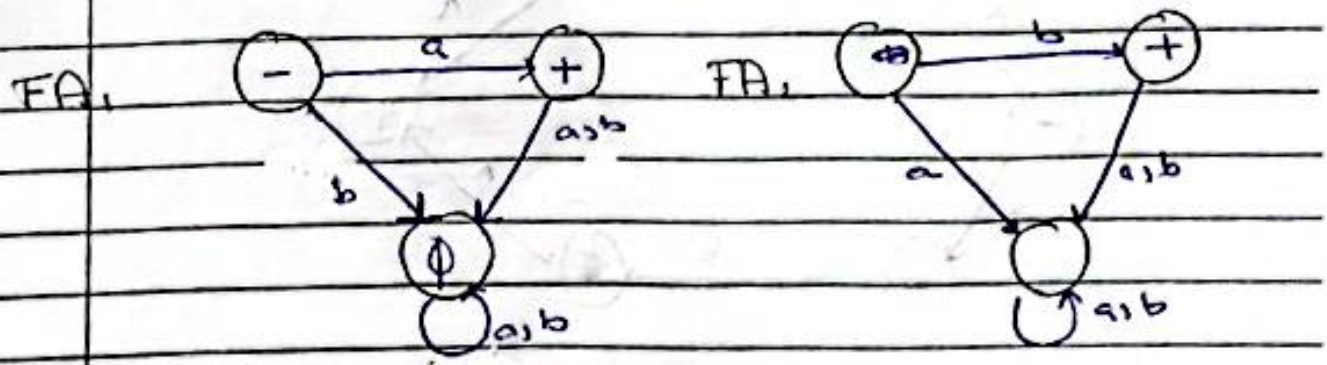
Day: Date: / /





$FA_1 + FA_2$, $FA_1 FA_2$, FA_1^*





GALAXAY

FA5

FA4

FA3

Day: Date: / /

RE = a + (a+b)* c

FA1 FA2

FA7 FA6

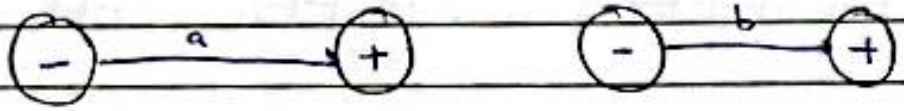
Thompsons construction Algorithm

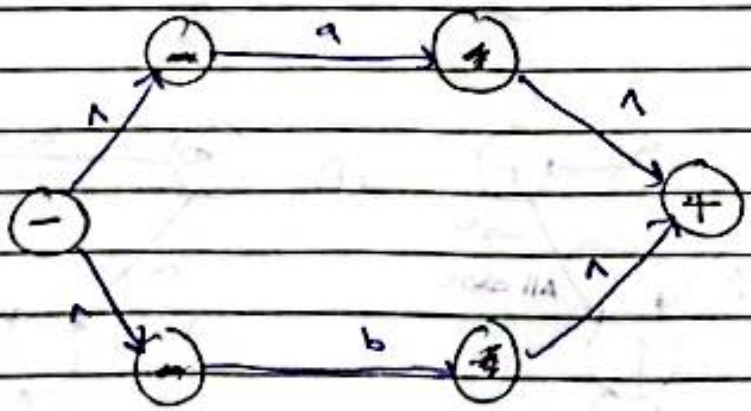
Create complete NFA of language

$r_1 = a + b$

→ Draw NFA for each term in r_1

→ Connect with ∧

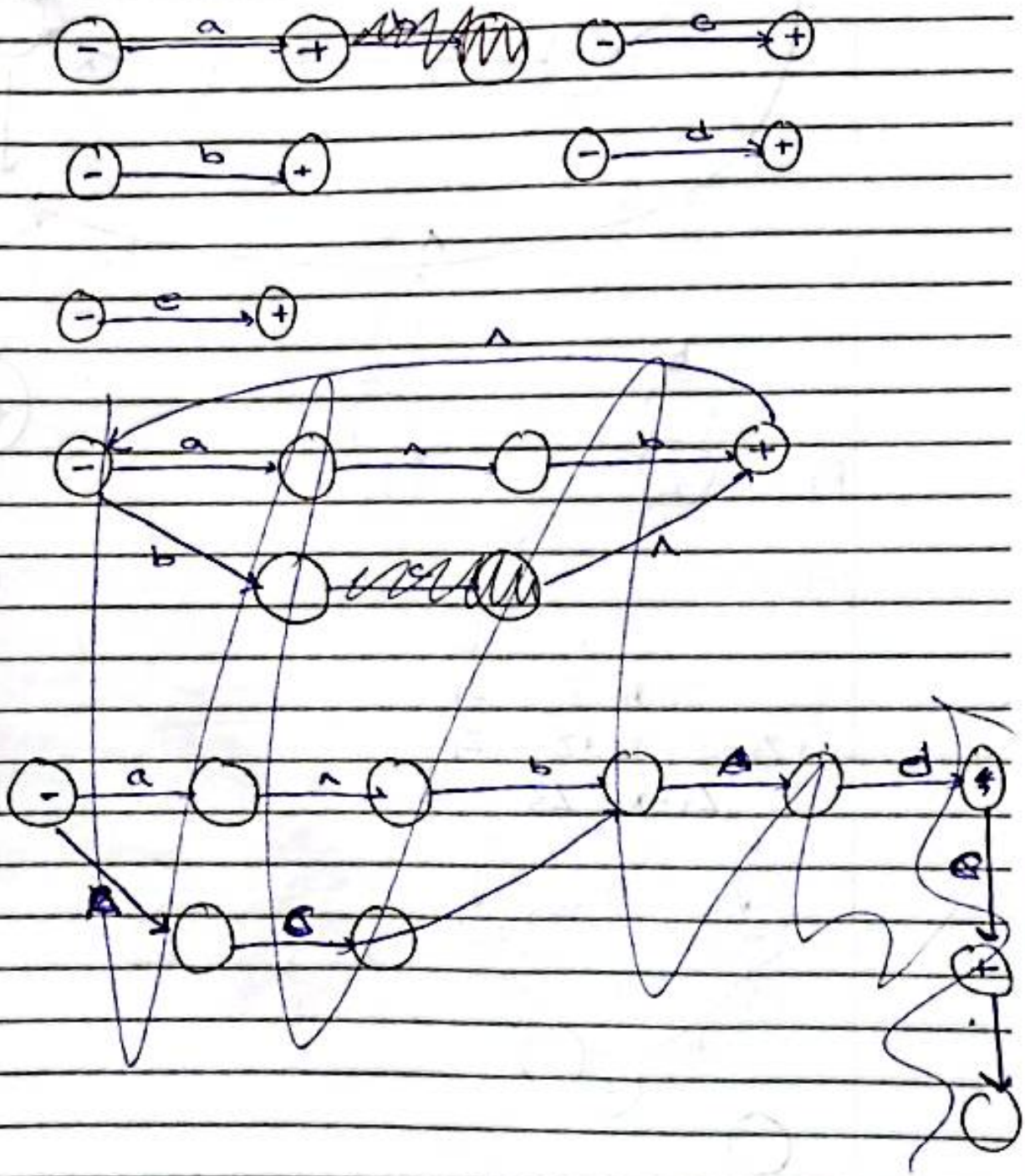




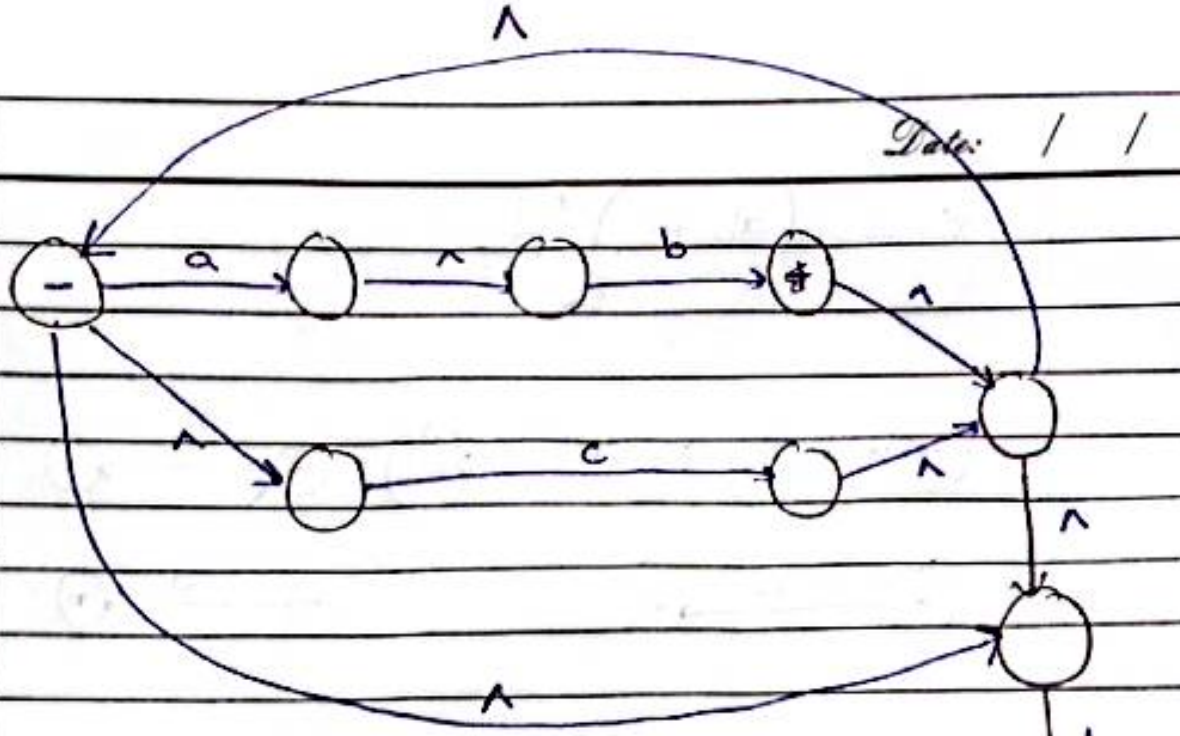
Day:

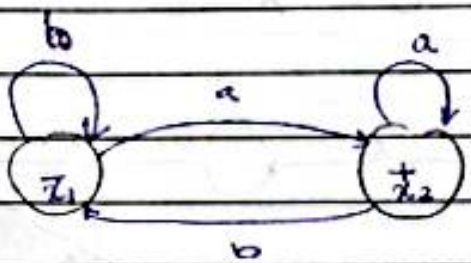
Date: / /

$Y = (ab+c)'de$



Day: Date: / /

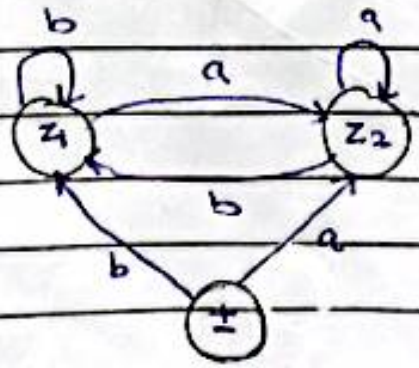


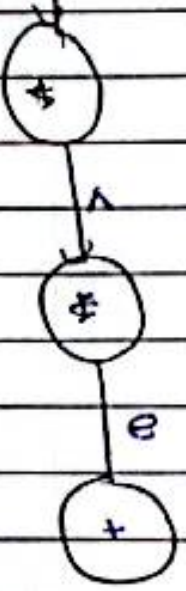


FA_1

$(FA_1)^*$:

	a	b
$Z_1 = q_1$	Z_2	Z_1
$+Z_2 = X_1 \text{ or } X_2 + Z_3$	Z_1	

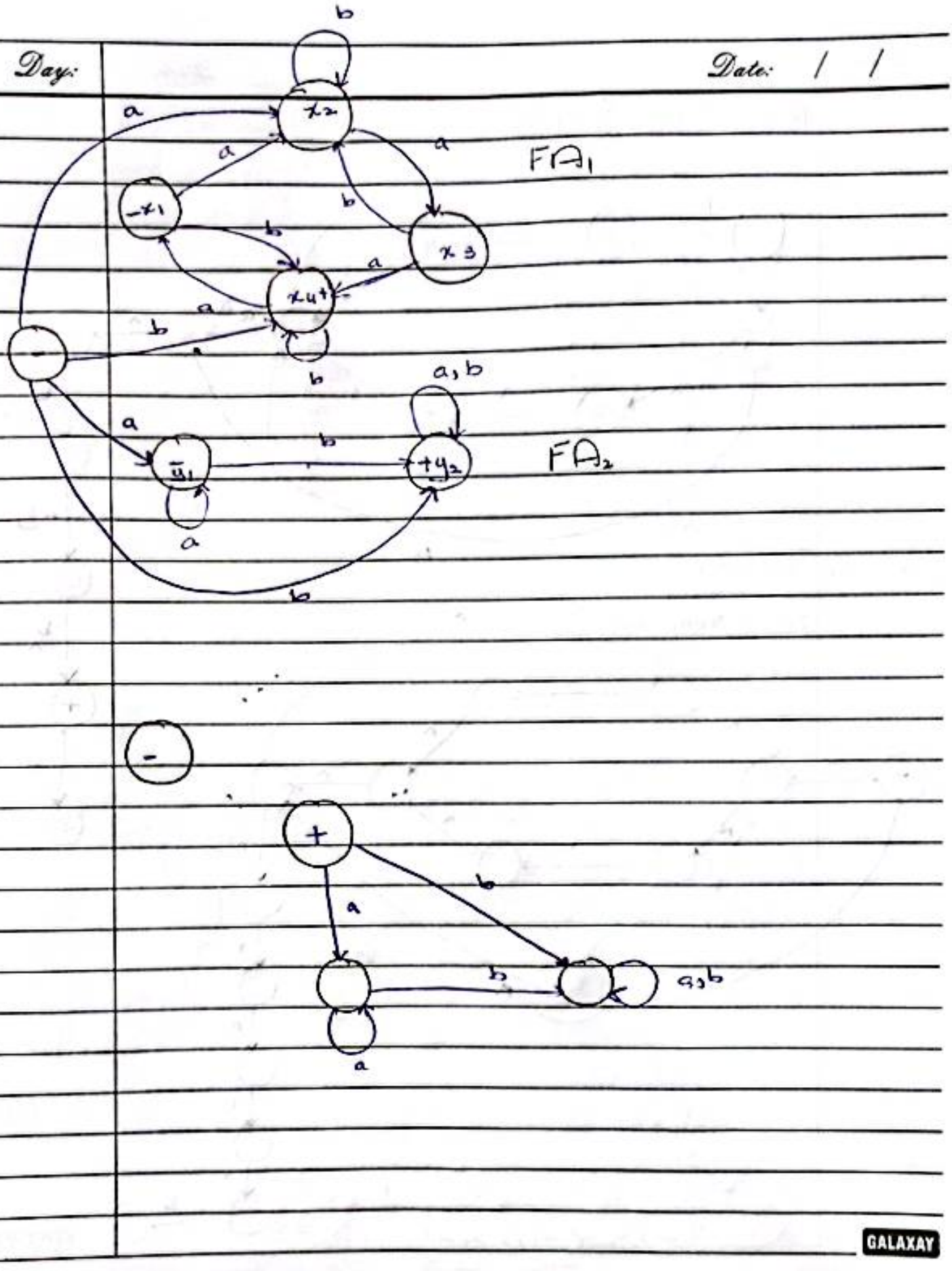




GALAXAY

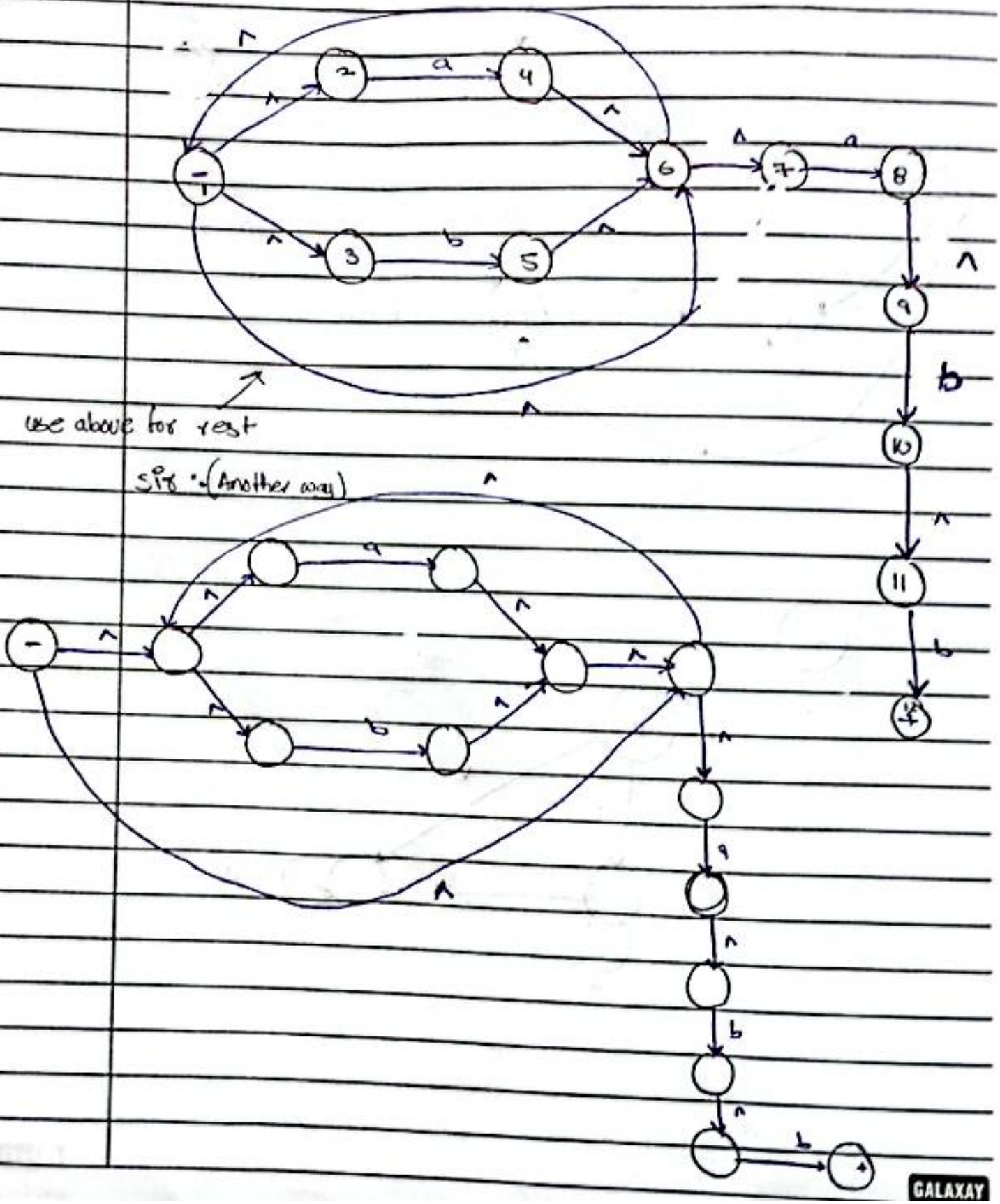
Day:

Date: / /



Day: Date: / /

R.E = $(a+b)^* abb$



GALAXAY

Day: Date: / /

ε-closure({1})

A = {1, 2, 3, 6, 7} → Read 'a' or 'b' at each state & create new set

write all states that can be reached on reading a from state 1. or set in above (black).

	a	b
A	B	C
B	B	D
C	B	C
D	B	+ε
+ε	B	C

move(A, a) = {4, 8}

ε-closure({4, 8})

B = {4, 6, 7, 1, 2, 3, 8, 9}

move(A, b) = {5}

ε-closure({5})

C = {1, 2, 3, 5, 6, 7}

move(ε, a) = {4, 8} = B

move(ε, b) = {5}

move(C, a) = {4, 8} = B

move(C, b) = {5}

move(D, a) = {4, 8}

move(D, b) = {5, 12}

ε-closure({5, 12}) = {5, 6, 1, 2, 3, 7, 12} = ε

move(B, a) = {8, 4}

ε-closure({4, 8}) = B

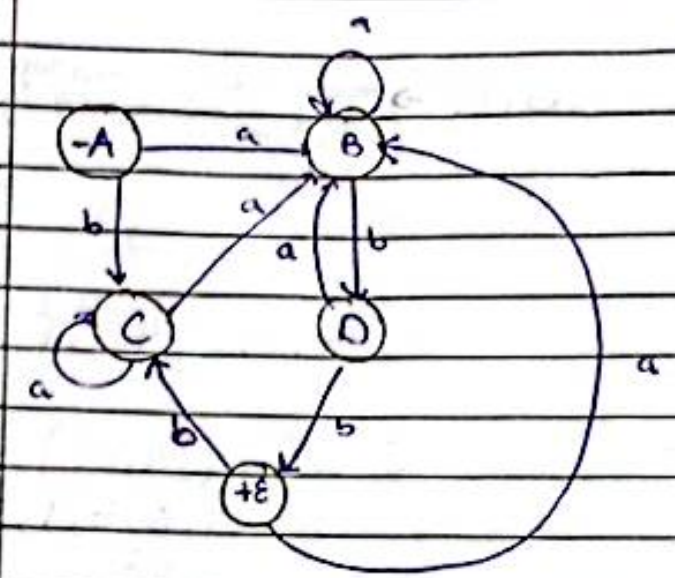
move(B, b) = {5, 10}

ε-closure({B, b})

D = {5, 6, 1, 2, 3, 7, 10, 11}

Day: Date: / /

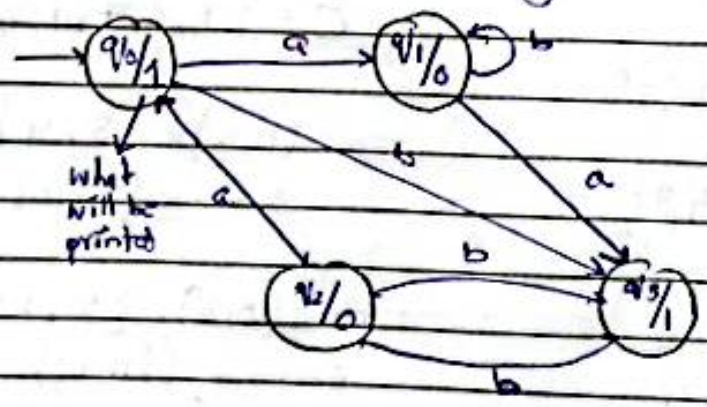
CHeck Hidden Markov Model!! (Extra for ML).



Moore Machine:

↳ Print something on moving states.

↳ No acceptance or rejection



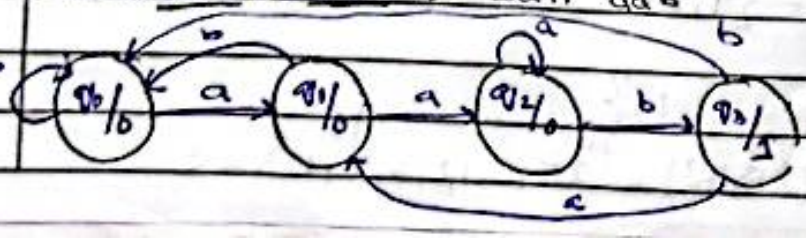
$\Sigma = \{a, b\}$

$\Gamma = \{0, 1\}$

Input: abab

10010

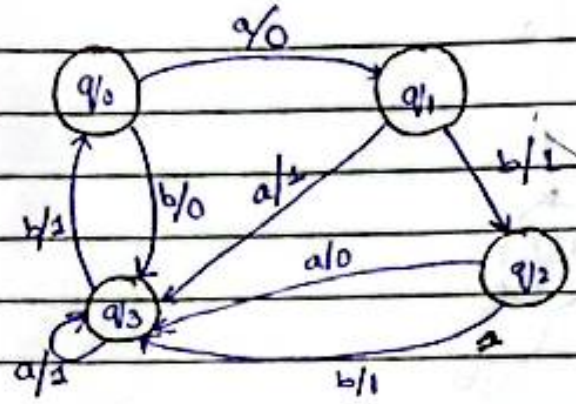
abbaabbbaab ⇒ Count 'aab'



GALAXAY

Days | CS-224 (FA with output) | Date: / /

Melay Machine



- Can have different output for incoming transitions
- Moore would give one output

- Output of Moore is one more than melay machine.
- No final state in both Melay & Moore

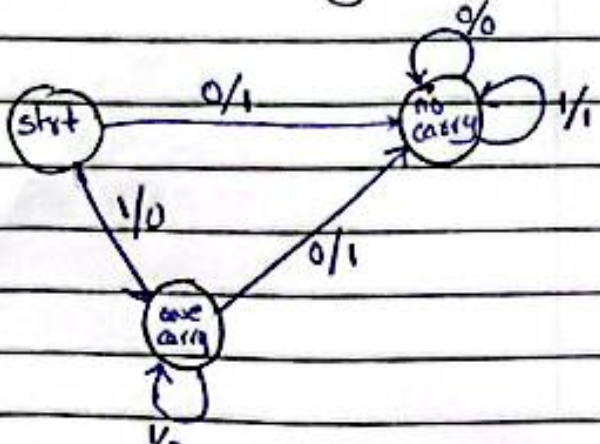
Melay: aaabb → 01110
Moore: aaabb → 001110

} can't say they accept same language on basis of output.

Build Machine that increments by 1.

$11+1 = 12$

1011
1
1100



HW
Design a Melay to decrement number by 1.

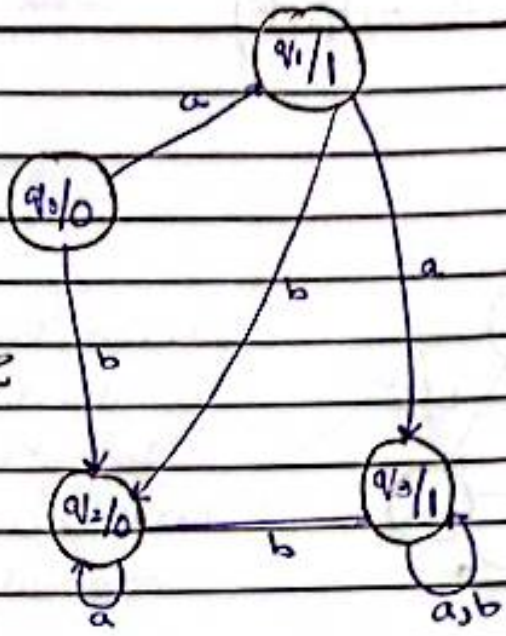
GALAXAY

Convert Moore to Mealy

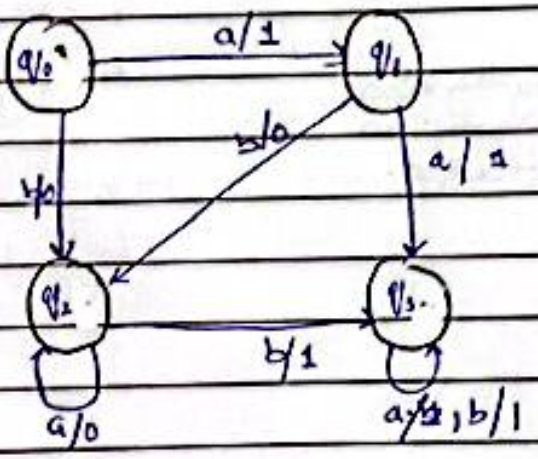
Day:

Date: / /

Moore

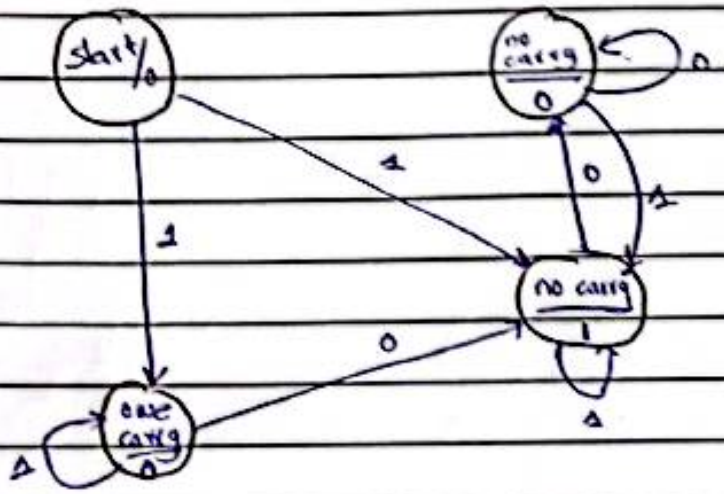


Mealy



→ move state's output onto incoming transition
→ remove output of those without incoming transitions

Day: Convert Mealy to Moore Date: / /



↳ Add 0 on start start
↳ Move output on incoming transitions onto state
↳ Separate the state with different output incoming transitions into same no. of outputs (distinct)
↳ Complete transitions.

Day: Date: / /

( ) → Precedence   < > → headers, templates

{ } → block   " ' ' → start/end of string

[ ] → Arrays

→ Every starting has an ending to be valid.

(()()()), ((())), ()

For regex:

You write either each case with | but then you need so many to cover all cases (infinite)

or use + or * but then can't guarantee the) to repeat same times as (.

$S \rightarrow \{S\} \mid \Lambda$ (→ non-terminals; terminals → 0/1)

$\Sigma = \{ \{, \} \}$ (letters; Alphabet)

CFG: created structure, unbounded / no upper limit / no idea when to stop (infinitely)

$S \rightarrow \{S\}$ (1)
$S \rightarrow \Lambda$ (2)
} production

→ Where you see S you can replace with right side.

Left side	Right side
non-terminals (can be replaced by production)	combination of terminals + non-terminals (can't be replaced)

$A \rightarrow \{S\}$ (1)

GALAXAY

Day:

Date: / /

Derivation

A → {S} ①
→ {Λ} ②
→ { }

A → {S} ①
→ {{S}} ③
→ {{Λ}} ②
→ {{{}}}

Production:

S → 0S1

S → SS

S → Λ

00100110 11 0011

match/group by most nearest

0 1
{ }

OR

S → 0S1 | SS | Λ

S → 0S1
⇒ 00S1
⇒ 00Λ11
⇒ 0011
(( ))

S → SS
⇒ 0S1S
⇒ 0Λ1S
⇒ 01 0S1
⇒ 01 0Λ1
⇒ 0101
( ) ( )

Do left to Right since we are doing left recursive process

GALAXAY

Day: Date: / /

S → (S) (S)

⇒ 0S1 S

⇒ 0 0S1 S 1

⇒ 00S1S1 S

⇒ 001 0S1S1 0S1

001 00S11S1 00S11

001 00 11S1 0011

00100 11 0S11 0011

00100 11 011 0011

Construct CFGs for unsigned ints

S → 0|1|2|3|4|5|6|7|8|9

S → S

S → SS

caps are non terminals

U → UD | D

D → 0|1|2|3|4|5|6|7|8|9

if it comes 0 it must come alone

Regex $(0+1+2+\cdots+9)^+$ → No restriction of zero at start so we can use Regex as well

GALAXAY

Day: Date: / /

CFG for a^*

$S \to Sa \mid \Lambda$ → Non-ambiguous

a^+

$S \to aS \mid a$

$S \to aS \mid a \mid \Lambda$ → Ambiguous

Sir k:

$S \to SS$

$S \to a$

$S \to \Lambda$

aa

$S \to SS$
$\Rightarrow aS$
$\Rightarrow aa$

$S \to SS$
$\Rightarrow SSS$
$\Rightarrow \Lambda SS$
$\Rightarrow \Lambda aS$
$\Rightarrow \Lambda aa$
$\Rightarrow aa$

$S \to SS$
$\Rightarrow SSS$
$\Rightarrow SSSS$
$\Rightarrow SSSSS$
⋮
when to stop?

If there multiple derivations to get 'aa' or other words then that CFG is Non-deterministic

$(a+b)^* aa (a+b)^*$

$S \to SaaS$
$S \to \Lambda$ X
$S \to aS$ X
$S \to bS$ X

$S \to XaaX$
$X \to aX$
$X \to bX$
$X \to \Lambda$

} Ambiguous

GALAXAY

Day: Date: / /

Palindrome

$S \to aSa \mid a$

$S \to bSb \mid b$

$S \to \Lambda$

$S \to S+S \mid S*S \mid a \mid b$

$S \to S+S$ $\quad a+a*b$

$\Rightarrow a+S*S$

$\Rightarrow a+a*S$

$\Rightarrow a+a*b$

$S \to S*S$ Yes! Ambigous

$S \to S+S*S$

$S \to a+S*S$

$S \to a+a*S$

$S \to a+a*b$

To find Ambiguity, use tree:

↳ Non-terminals → Nodes

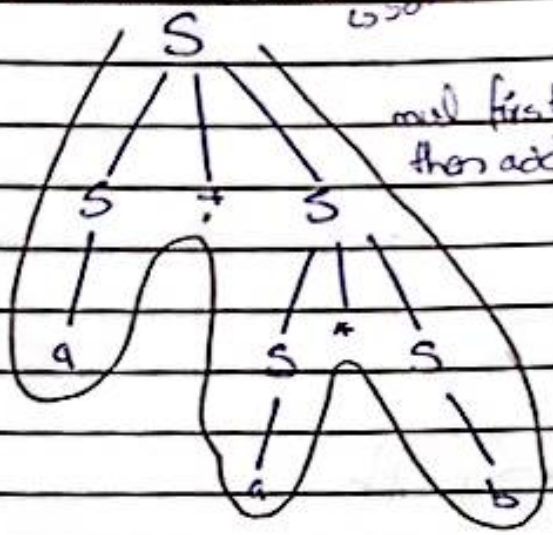
↳ Terminals → leaf

GALAXAY

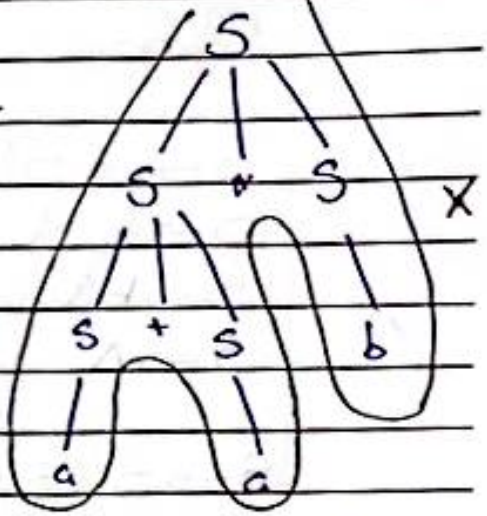
Day:

Parse tree method → To make derivation process easier (Syntactic Analysis)

Date: / /



mul first then add ✓



X

Add first then mul

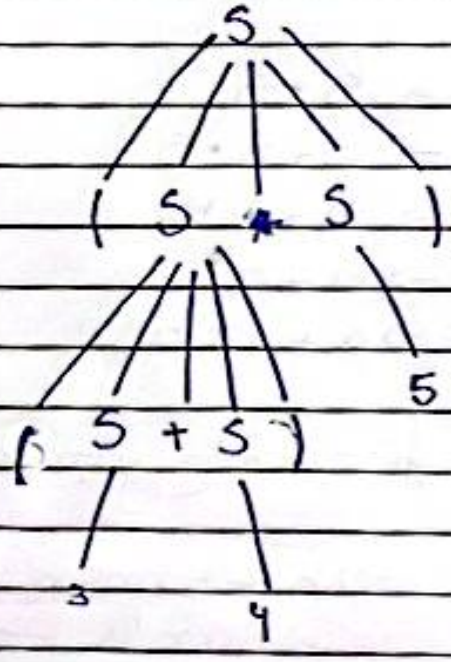
a + a * b
2 2 3

PEMDAS

D

S → (S+S)
S → (S*S)
S → 0|1|2|...|9

(3+4)*5)



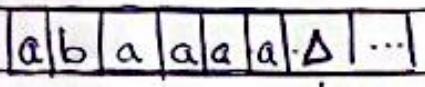
GALAXAY

Day:

Date: / /

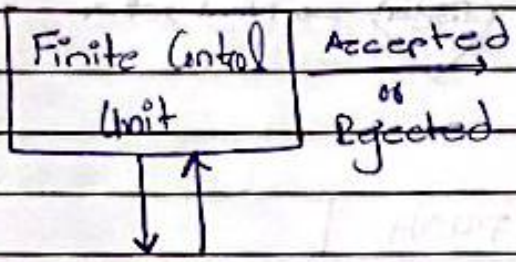
Push Down Automata (PDA)

Input Tape
↳ A sequence of blocks (infinite)



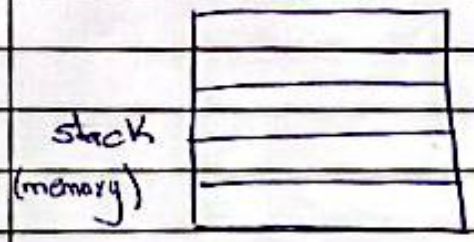
Not in Σ - {input letters or terminals}

↳ End of input tape



$\Gamma = \{ .. \}$

↳ A set of symbols that can be pushed or popped from stack



$\delta = \{ .. \}$

↳ Set of transitions to move from one state to another OR transition function

Day:

Date: / /

START

Accept

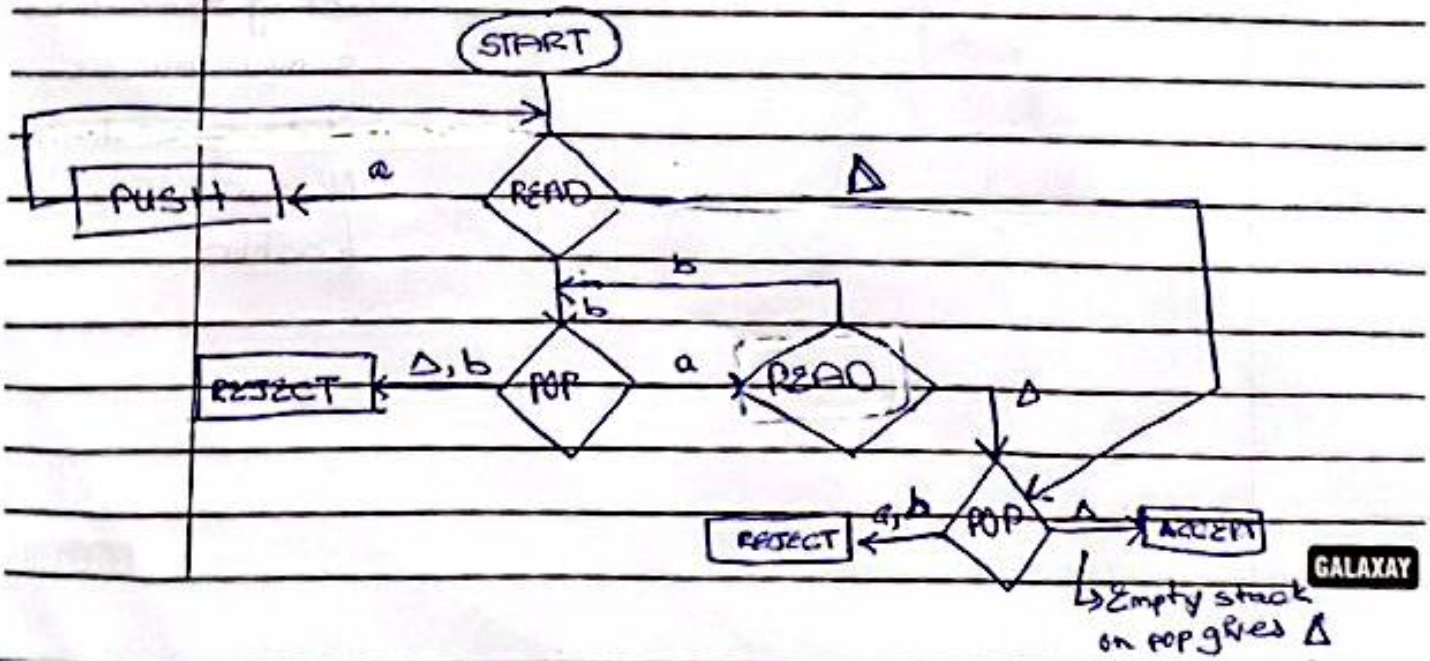
Reject

Read

PUSH

POP

Language : $a^n b^n$



GALAXAY

Date: / /

Language: $a^n b^{2n}$

Chomsky Normal form:

↳ Right side of Production has (Refer Notes)
either two non-terminals or one terminal

$S \rightarrow AA \quad S \rightarrow a$

Convert CFG to above

Step 1. Remove Null Production

$A \rightarrow \Lambda$

Exp

$S \rightarrow aX$

$X \rightarrow \Lambda$

This doesn't add anything to the language. Just an overhead

↳ Replace X with Λ everywhere it appears

$S \rightarrow aX \mid a$

else remains the same.

If $X \rightarrow \Lambda$ is removing aX would not generate ab

GALAXAY

Day: Exp Date: / /

$S \to aSb \mid bSb \mid \Lambda$

Find all Λ productions

$S \to \Lambda$

Put Λ at S where it appears and add result as a new production

$S \to aSa \Rightarrow a\Lambda a \Rightarrow aa$

$S \to bSb \Rightarrow b\Lambda b \Rightarrow bb$

Remove Λ productions & add new ones

$S \to aSa \mid bSb \mid bb \mid aa$

Exp:

$S \to ABAC$

$A \to aA \mid \Lambda$

$B \to bB \mid \Lambda$

$C \to c$

$S \to ABAC \mid BAC \mid ABC \mid BC \mid AAC \mid AC \mid C$

$A \to aA \mid a$

$B \to bB \mid b$

$C \to c$

$A \to \Lambda$

$B \to \Lambda$

$S \to ABAC \Rightarrow AAAC \Rightarrow AAC$

$S \to ABAC \Rightarrow \Lambda BAC \Rightarrow BAC \Rightarrow AC$

$A \to aA \Rightarrow a\Lambda \Rightarrow a$

$B \to bB \Rightarrow b\Lambda \Rightarrow b$

ABAC ⇒ AAAC ⇒ AAC ⇒ AC

Page: Date: / /

$S \to a \mid Xb \mid aYa$
$X \to Y \mid \Lambda$
$Y \to b \mid \Lambda$

$X \to \Lambda$
$Y \to \Lambda$

$S \to Xb$ Replacing Y with Λ gives Λ as new production
$\Rightarrow \Lambda b$ So, we this production all
$\Rightarrow b$ together

$S \to a \mid Xb \mid aYa \mid b \mid aa$
$X \to Y$
$Y \to b$

Step 1. Identify Unit Productions & Remove them

Exp

$S \to A \mid bb$ ✓ Those production which
$A \to B \mid b$ can be replaced by
$B \to S \mid a$ a sing non-terminal
non-t $\to$ single non-t
such are unit
productions

$S \to A \to b$ (over head)
so
$S \to b$

GALAXAY

Day: Date: / /

Step 1. Identify all UP

S → A

A → B

B → S

Step 2: Replace

S → A

⇒ b ⇒ A

⇒ B

⇒ a

remove

S → ~~A~~ | bb | b | a

A → B | b

B → S | a

A → B

⇒ a ⇒ A → B → S

⇒ bb | b | a

check on this

Now,

S → bb | b | a

A → ~~A~~ | b | a | bb

B → ~~S~~ | a | bb | b

B → S

⇒ bb ⇒ b

GALAXAY

Day: Date: / /

Exp: Remove UP

$S \to XY$

$X \to a$

$Y \to Z \mid b \Rightarrow Y \to a$

$Z \to M \Rightarrow Z \to a$

$M \to N \Rightarrow M \to a$

$N \to a$

$S \to XY$
$X \to a$
$Y \to a \mid b$
} In CNF as $Y \to a$, $Y \to b$

$Z \to a$
$M \to a$
$N \to a$
} Remove this

$S \to aSa \mid bSb \mid a \mid b \mid aa \mid bb$

Step 03:

All those production with combinations of terminal & non-terminals or more than one terminals. Replace them (terminals) with a seperate production $A \to a$ (Replace small with Cap & Cap → small)

$S \to aSa$ } Replace by
$S \to ASA$ }
$A \to a$

GALAXAY

Day: Date: / /

$S \to bSb$ $S \to aa$ $S \to bb$

$S \to BSB$ $S \to AA$ $S \to BB$

$B \to b$

Now,

$S \to ASA \mid BSB \mid a \mid b \mid AA \mid BB$

$A \to a$

$B \to b$

All those productions with / terminals > 2.

keep first & replace rest with a new terminal

$S \to ASA$ $S \to BSB$

$S \to AR_1$ $S \to BR_2$

$R_1 \to SA$ $R_2 \to SB$

Now,

$S \to AR_1 \mid BR_2 \mid a \mid b \mid AA \mid BB$

$A \to a$

$B \to a$

$R_1 \to SA$

$R_2 \to SB$

Exp: $S \to bA \mid aB$

$A \to bAA \mid aS \mid a$

$B \to aBB \mid bS \mid b$

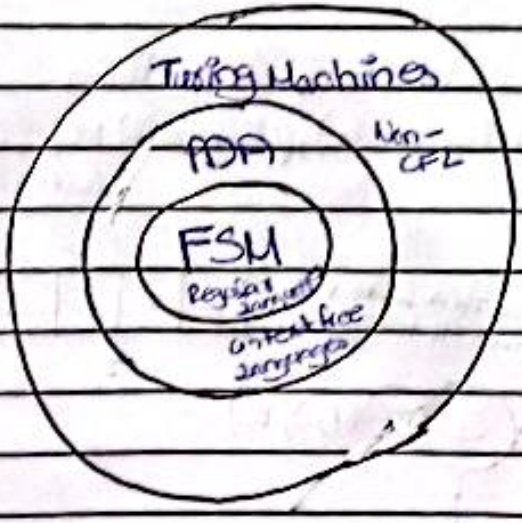
GALAXAY

Day: Date: / /

S → BA | AB
A → BAA | AS | a
B → ABB | BS | b

A → BR1 B → AR2
R1 → AA R2 → BB

Now,
S → BA | AB
A → BR1 | AS | a
B → AR2 | BS | b
R1 → AA
R2 → BB



Context free:
↳ Don't consider what left to right of a letter.

Eg $a^n b^n c^n$
↳ Can't do it by PDA
↳ Need more memory
↳ TMs have infinite memory

Day: Date: / /

TAPE

- Only memory in TMs
- Read / Write here
- Tape head can move Left/Right

a	b	a	a	␣	␣	...

↳ Empty (Δ)

$\Sigma = \{a, b\}, \quad \Gamma = \{a, \dots\}$

↳ Read from tape

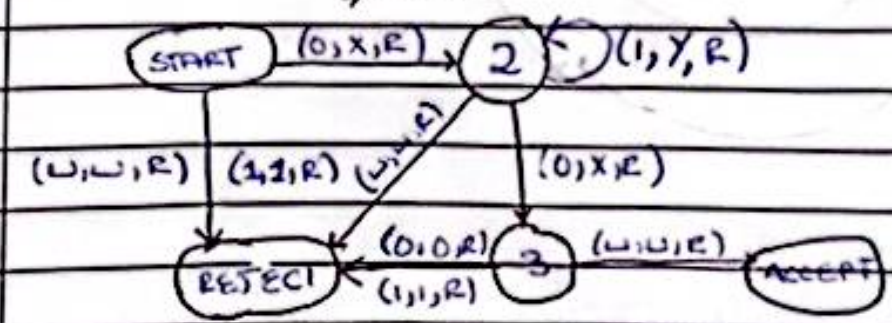
↳ Write to tape

Finite set of states

- Transition :- Rules, Program, Algorithm
- (letter (read), letter (write), (Movement) M_{Left} (Left or Right))

$L = 01^*0$ Read 0 at THA write X then move TH to right

0	1	0	␣	...

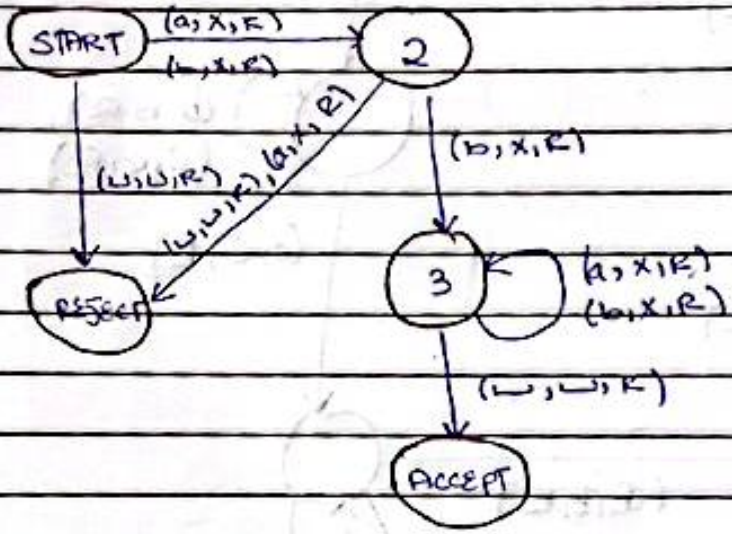


GALAXAY

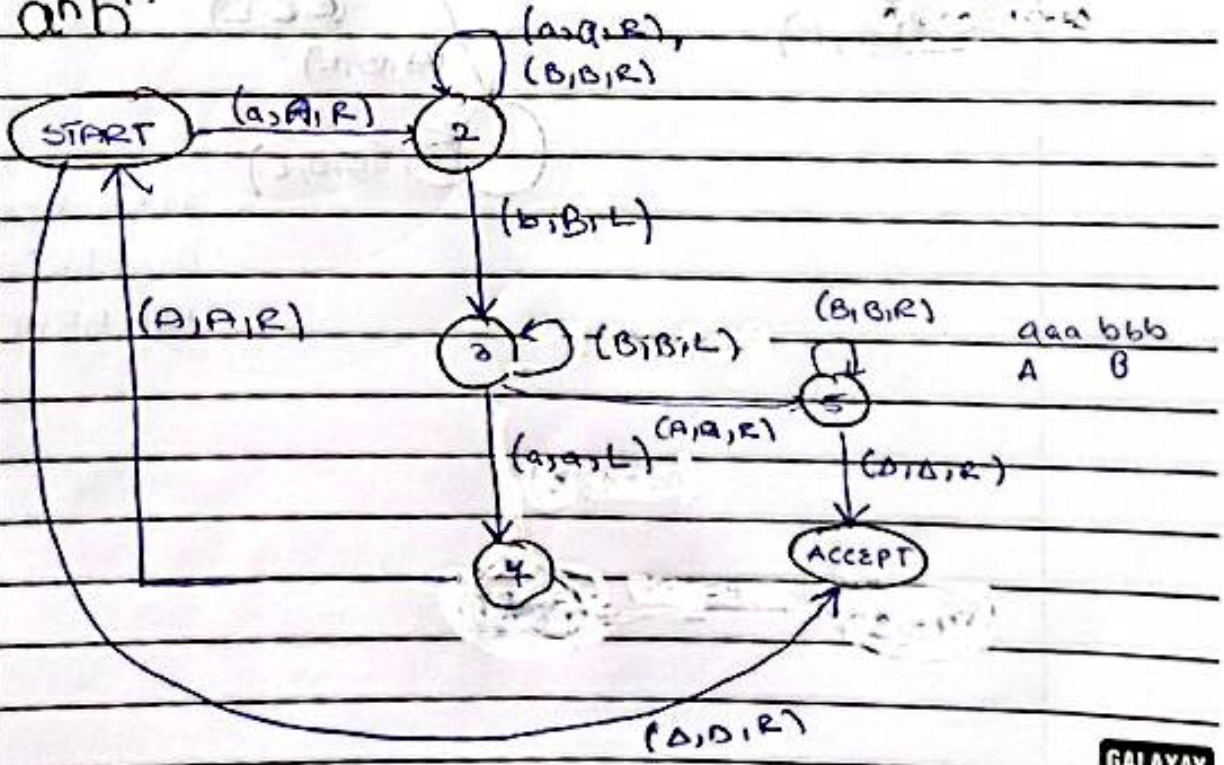
Day: Date: / /

Eg: $L = a^n b^n$

$(a+b)\,b\,(a+b)^*$

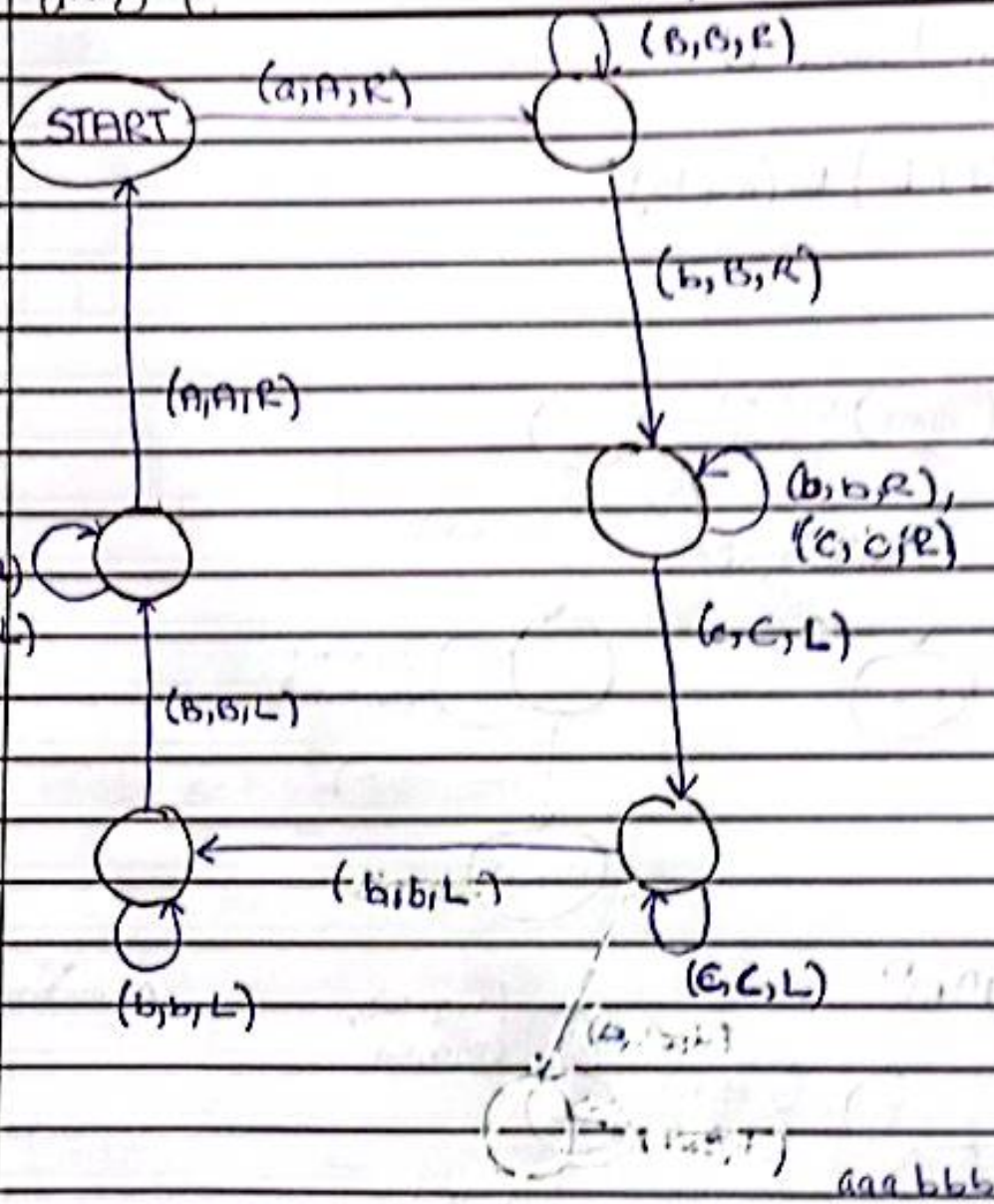


$a^n b^n$



GALAXAY

Day: $a^n b^n c^n$ Date: / /



aaabbbccc

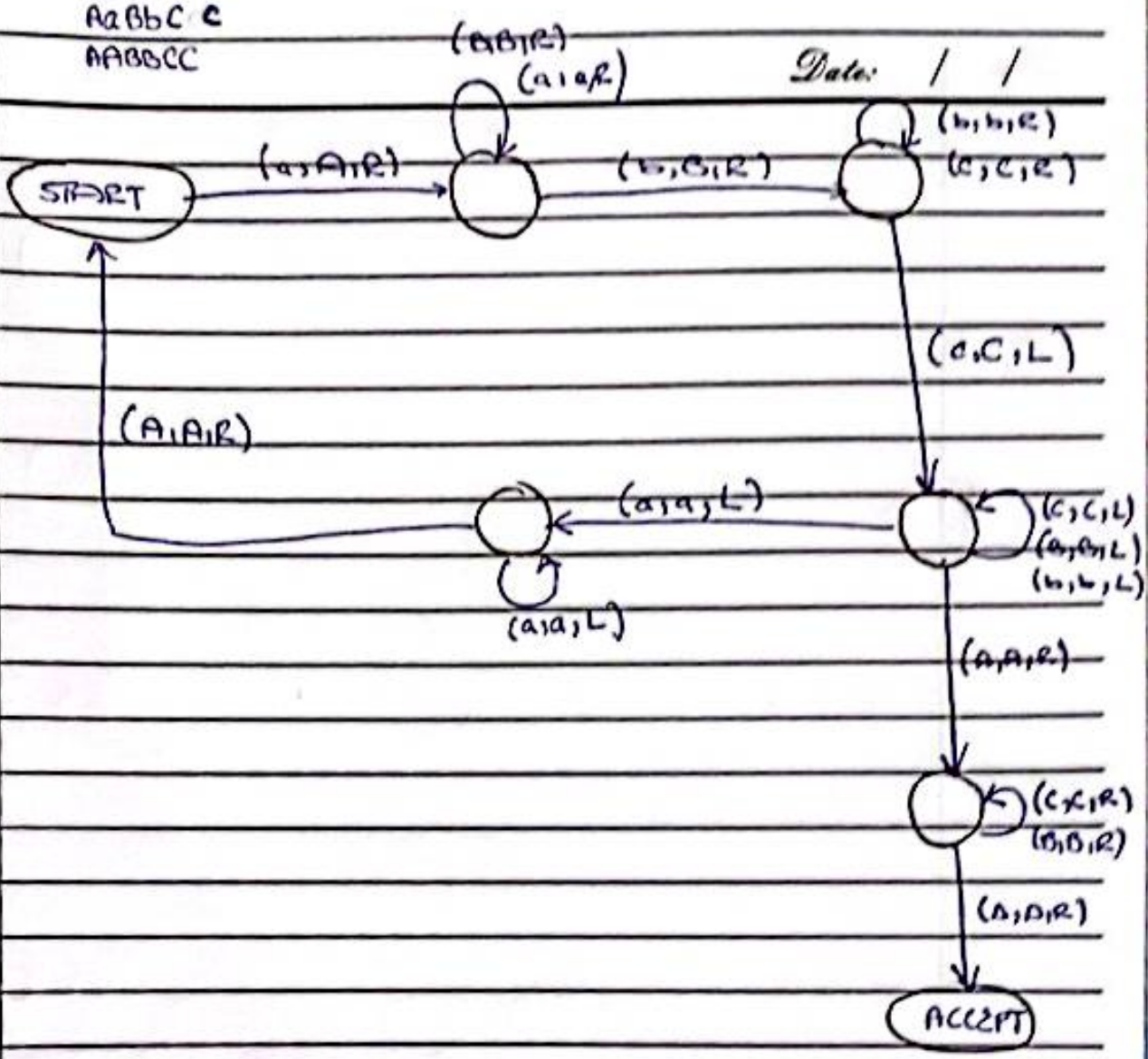
AaaBbbCcc

AAaBBbCCc

aabbcc
AaBbC c
AABBCC

Day:

Date: / /



GALAXAY